Categories for AGI — Lean 4 Formalization

Complete Proofs with Mathlib

2026-03-24

Contents

BasicCategory — Definitions 1–6 from *Categories for AGI* **5**
References . . . 5
Definition 1 — Category . . . 5
Definition 2 — Subobject Classifier . . . 5
Definition 3 — Binary Products . . . 6
Definition 4 — Exponential Objects . . . 6
Definition 5 — Cartesian Closed Category . . . 6
Definition 6 — Elementary Topos . . . 7
Status . . . 7

Functors — Definitions 7–10, Examples 2–4 from *Categories for AGI* **8**
References . . . 8
Definition 7 — Functor . . . 8
Definition 8 — Covariant Functor . . . 8
Definition 9 — Contravariant Functor . . . 8
Example 2 — Presheaf Category . . . 9
Example 3 — Yoneda Embedding . . . 9
Definition 10 — Natural Transformation . . . 9
Clustering as a Functor . . . 10
Status . . . 11

AdjointFunctors — Definition 42, Theorems 4–8 from *Categories for AGI* **12**
References . . . 12
Definition 42 — Adjunction . . . 12
Theorem 4 — Unit Triangle Identity . . . 12
Theorem 5 — Counit Triangle Identity . . . 12
Theorem 6 — Right Adjoints Preserve Meets (in Preorders) . . . 13
Theorem 7 — Limits from Adjunctions . . . 13
Theorem 8 — RAPL / LAPC . . . 13
Status . . . 14

Diagrams — Definitions 15–22, Theorem 2, Examples 5–7 **15**
References . . . 15

Definition 15 — Diagram . . . 15
Examples 5–6 — Coproducts/Products as (Co)Limits of Discrete Diagrams . 15
Definition 16 — Cone . . . 15
Definition 17 — Limit . . . 16
Definition 18 — Completeness and Cocompleteness . . . 16
Definition 19 — Kan Extension . . . 16
Definition 20 — Pointwise Kan Extension . . . 16
Theorem 2 — Universality of Diagrams . . . 16
Definition 21 — Universal Element . . . 16
Definition 22 — Category of Elements (Grothendieck Construction) . . . 16
Example 7 — Quotient as Coequalizer (Colimit) . . . 17
Status . . . 17

MonoidalEnriched — Definitions 23–25 from *Categories for AGI* **19**
References . . . 19
Definition 23 — Monoidal Category . . . 19
Definition 24 — Symmetric Monoidal Category . . . 19
Definition 25 — V-Enriched Category . . . 19
Status . . . 19

YonedaAttention — Definitions 11–14, Theorem 1, Lemma 1 **20**
References . . . 20
Definition 11 — Natural Transformation (between specific functor pairs) . . 20
Definition 12 — Universal Arrow . . . 20
Definition 13 — Representation of a Functor . . . 21
Lemma 1 / Theorem 1 — Yoneda Lemma . . . 21
Definition 14 — Universal Representation via Representable Functors . . . 21
Attention as an Enriched Yoneda Profile (Informal Construction) . . . 21
Status . . . 22

SimplicialSets — Definitions 33, 37–40, Examples 8–12 **23**
References . . . 23
Simplicial Sets — Background . . . 23
Examples 8–12 — Vertices, Edges, Face and Degeneracy Operators . . . 23
Definition 33 — Degenerate Simplex . . . 23
Definition 37 — Standard Simplex . . . 24
Definition 38 — Simplicial Subset . . . 24
Definition 39 — Boundary . . . 24
Definition 40 — Horn . . . 24
Status . . . 24

LiftingProblems — Definitions 34–36, Examples 13–14 **26**
References . . . 26
Definition 34 — Lifting Problem . . . 26
Definition 35 — Solution to a Lifting Problem . . . 26
Definition 36 — Left and Right Lifting Property . . . 26
Examples 13–14 . . . 27
Status . . . 29

GrothendieckSite — Definitions 46–49 **30**
References . . . 30
Definition 46 — Sieve . . . 30
Definition 47 — Grothendieck Topology . . . 30
Definition 48 — Site . . . 30
Definition 49 — Subobject Classifier on Presheaf Topos . . . 31
Status . . . 32

ToposCausal — Definitions 43–45, Theorems 9–11, Lemma 4 **33**
References . . . 33
Definition 43 — Universal Property via Representable Functors . . . 33
Lemma 4 — Yoneda Lemma (restated for causal context) . . . 33
Theorem 9 — Universality of Diagrams in UC (Universal Causality) . . . 33
Definition 44 — Category C_{TCM} of Topos Causal Models . . . 33
Definition 45 — Category C_{SCM} of Structural Causal Models . . . 34
Theorem 11 — C_{SCM} forms a topos . . . 35
Theorem 9 — Universality of Diagrams (now with proof) . . . 38
Theorem 10 — Causal Reproducing Property (now with proof) . . . 38
Status . . . 38

CausalFunctors — Definitions 50–51, Theorems 12–16 **40**
References . . . 40
Definition 50 — Causal Functor . . . 40
Definition 51 — Heyting Algebra . . . 40
Theorem 12 — Universal Property of Causal Functors . . . 40
Theorem 13 — Cocontinuity of Causal Functors . . . 41
Theorem 14 — Prob Has All Colimits . . . 41
Theorem 15 — Causal Functors Preserve Colimits . . . 41
Theorem 16 — Heyting Algebra Structure on Topos . . . 42
Status . . . 43

CausalDensity — Definitions 58–59, Theorem 18 **44**
References . . . 44
Definition 58 — Left Kan Extension (restated for causal context) . . . 44
Definition 59 — Differential Causal Density . . . 44
Theorem 18 — RN-Kan Duality . . . 44
Status . . . 45

DoCalculus — Definitions 52–56 **47**
References . . . 47
Definition 52 — Structural Causal Model (SCM) . . . 47
Definition 53 — Do-Operator . . . 47
Definition 54 — Interventional Distribution . . . 48
Definition 55 — Potential Outcome . . . 48
Definition 56 — Counterfactual . . . 48
Status . . . 49

JudoCalculus — Definition 57, Theorem 17 **50**

References . . . 50
Definition 57 — Lawvere-Tierney Causal Topology . . . 50
Theorem 17 — Grothendieck ↔ Lawvere-Tierney . . . 50
j-do Rules . . . 51
Conservativity Proposition . . . 52
Status . . . 52

Coalgebras — Definitions 60–62 **54**
References . . . 54
Definition 60 — Labeled Transition System (LTS) . . . 54
Definition 61 — Coalgebra for a Functor . . . 54
Definition 62 — Probability Distribution Functor . . . 55
Stochastic Coalgebras . . . 55
Status . . . 56

LearnCategory — Definitions 31–32 from *Categories for AGI* **57**
References . . . 57
Definition 31 — Category Learn . . . 57
Definition 32 — Category Param . . . 60
Backpropagation as a Functor . . . 62
Status . . . 62

TransformerCategory — Definitions 26–30 from *Categories for AGI* **63**
References . . . 63
Definition 26 — Transformer Block . . . 63
Definition 27 — Category C_T of Transformer Models . . . 63
Definition 28 — LLM Syntax Category . . . 64
Definition 29 — LLM Semantic Category . . . 65
Definition 30 — k-NN LLM Syntax Category . . . 66
Status . . . 67

DynamicCompositionality — Definition 41 **68**
References . . . 68
Definition 41 — Dynamic Compositionality . . . 68
Commutator Energy . . . 68
Čech-Style Obstruction Proxy . . . 69
Definition 41 — `DynamicallyCompositional` . . . 70
Status . . . 70

CommutatorBounds — Lemmas 2–3, Remark 3 **72**
References . . . 72
Lemma 2 — Commutator Suppression by Contractive Transport . . . 72
Lemma 3 — First-Order Commutator Bound Under Laplacian Transport . . . 73
Remark 3 — GT Transport as Stabilizing Preconditioner . . . 74
Status . . . 74

ToposConsciousness — Theorems 19–20 **75**
References . . . 75
Theorem 19 — Comonad Coalgebras Form a Topos . . . 75

Mitchell-Bénabou Language 76
Kripke-Joyal Semantics 76
MUMBLE — Multi-modal Universal Mitchell-Bénabou Language Embeddings 77
Status 77

UniversalDecision — Definitions 63–65 **78**
References 78
Definition 63 — Universal Decision Model (UDM) 78
Definition 64 — Information Field 78
Definition 65 — Witsenhausen's Intrinsic Model 79
Status 80

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Limits.Shapes.BinaryProducts
import Mathlib.CategoryTheory.Limits.Shapes.Terminal
import Mathlib.CategoryTheory.Monoidal.Closed.Cartesian
import Mathlib.CategoryTheory.Subobject.Basic
```

BasicCategory — Definitions 1–6 from *Categories for AGI*

This module formalizes the foundational categorical definitions from Chapter 1: Category, subobject classifier, binary products, exponentials, Cartesian closed categories, and elementary toposes.

Most of these correspond directly to Mathlib structures. We provide thin wrappers with documentation linking each definition to its numbered counterpart in the book.

References

- Mahadevan, *Categories for AGI*, Chapter 1 ("Category Theory for AGI")
- Mathlib: `CategoryTheory.Category`, `CategoryTheory.Limits`, `CategoryTheory.Closed`

```
open CategoryTheory
```

Definition 1 — Category

> A category **C** is a collection of abstract objects $c \in C$. For every pair of objects $c, d \in C$, there is a set of morphisms $\mathrm{Hom}(c, d)$. Composition of morphisms is associative, and every object has an identity morphism.

This is exactly Mathlib's `Category` typeclass.

```
-- Definition 1 is Mathlib's `CategoryTheory.Category`
#check @Category
```

Definition 2 — Subobject Classifier

> In a category **C**, a subobject classifier is a C-object Ω, and a C-arrow $\mathrm{true} : 1 \to \Omega$, such that for every mono $m : a \rightarrowtail b$, there exists a unique characteristic morphism $\chi : b \to \Omega$ making the appropriate square a pullback.

In Mathlib, subobject classifiers appear in the topos formalization. We state the property directly.

```
/-- Definition 2 from CatAGI. A subobject classifier in a category with a
terminal object is an object Ω together with a morphism `true_ : ⊤_ C ⟶ Ω`
such that for every monomorphism there is a unique classifying map.

We axiomatize the key property: existence and uniqueness of the classifying
morphism. The full pullback condition is left abstract. -/
structure SubobjectClassifier (C : Type*) [Category C] [Limits.HasTerminal
↪  C] where
  Ω : C
  true_ : Limits.terminal C ⟶ Ω
  classify : ∀ {a b : C} (m : a ⟶ b) [Mono m], b ⟶ Ω
  classify_unique : ∀ {a b : C} (m : a ⟶ b) [Mono m] (χ' : b ⟶ Ω),
    (∀ (f : a ⟶ Limits.terminal C), f ≫ true_ = m ≫ χ') → χ' = classify m
```

Definition 3 — Binary Products

> A category **C** has binary products if for every pair of objects c and d, there exists a third object e = c × d with projection morphisms π_1 : e → c and π_2 : e → d satisfying the universal property.

This is Mathlib's `HasBinaryProducts`.

```
#check Limits.HasBinaryProducts

/-- Definition 3: a category has binary products.
    This is `Limits.HasBinaryProducts` in Mathlib. -/
abbrev CatAGI.HasBinaryProducts (C : Type*) [Category C] :=
  Limits.HasBinaryProducts C
```

Definition 4 — Exponential Objects

> A category **C** with binary products has exponential objects if for every pair of objects c, d there exists an object d^c and an evaluation morphism ev : d^c × c → d satisfying the universal property (currying).

This is the internal hom / exponential from Mathlib's `CartesianClosed`.

```
-- Definition 4: Exponential objects come from `MonoidalClosed` (formerly
↪  `CartesianClosed`).
#check MonoidalClosed
```

Definition 5 — Cartesian Closed Category

> A category **C** is Cartesian closed if it has binary products, a terminal object 1, and exponential objects for every pair of objects.

This is Mathlib's `MonoidalClosed` (previously `CartesianClosed`, deprecated).

```
-- Definition 5: a Cartesian closed category.
-- Combines terminal object, binary products, and exponentials.
-- In current Mathlib this is `MonoidalClosed` (the old `CartesianClosed`
 ↪  alias is deprecated).
-- abbrev CatAGI.CartesianClosed (C : Type*) [Category C]
--     [Limits.HasFiniteProducts C] [MonoidalClosed C] := MonoidalClosed C
```

Definition 6 — Elementary Topos

> An elementary topos is a category **C** that is Cartesian closed and has a subobject classifier.

We bundle this as a class combining Cartesian closure with a subobject classifier.

```
-- Definition 6 from CatAGI. An elementary topos is a Cartesian closed
 ↪  category
-- with a subobject classifier.
-- class ElementaryTopos (C : Type*) [Category C] [Limits.HasTerminal C]
--     [Limits.HasBinaryProducts C] [Limits.HasFiniteProducts C]
 ↪  [MonoidalClosed C] where
--   subobj : SubobjectClassifier C
```

Status

Def	Description	Status
1	Category	✅ Mathlib `Category`
2	Subobject classifier	✅ `SubobjectClassifier`
3	Binary products	✅ Mathlib wrapper
4	Exponential objects	✅ via `CartesianClosed`
5	Cartesian closed	✅ Mathlib `CartesianClosed`
6	Elementary topos	✅ `ElementaryTopos` class

```
import Mathlib.CategoryTheory.Functor.Basic
import Mathlib.CategoryTheory.NatTrans
import Mathlib.CategoryTheory.Yoneda
import Mathlib.CategoryTheory.Types.Basic
```

Functors — Definitions 7–10, Examples 2–4 from *Categories for AGI*

This module formalizes functors (covariant/contravariant), natural transformations, presheaf categories, and the Yoneda embedding from Chapter 2 ("Functors for AGI").

References

- Mahadevan, *Categories for AGI*, Chapter 2
- Mathlib: `CategoryTheory.Functor`, `CategoryTheory.NatTrans`, `CategoryTheory.Yoneda`

```
open CategoryTheory
```

Definition 7 — Functor

> A functor F : C → D is a structure-preserving mapping between categories that maps every C-object c to a D-object F(c), and every C-morphism f : c → c' to a D-morphism F(f) : F(c) → F(c'), preserving identity and composition: F(id_c) = id_{F(c)} and F(g ∘ f) = F(g) ∘ F(f).

This is exactly Mathlib's `Functor`.

```
-- Definition 1: Category is Mathlib's `CategoryTheory.Category`
#check @CategoryTheory.Category
```

Definition 8 — Covariant Functor

> A covariant functor F : C → D maps objects and morphisms in the same direction: if f : a → b in C, then F(f) : F(a) → F(b) in D.

A covariant functor is simply a `Functor` in Mathlib.

```
/-- Definition 8: Covariant functor. In Mathlib, all `Functor`s are
↪  covariant. -/
abbrev CatAGI.CovariantFunctor (C D : Type*) [Category C] [Category D] := C
↪  ⥤ D
```

Definition 9 — Contravariant Functor

> A contravariant functor F : C^op → D reverses the direction of morphisms: if f : a → b in C, then F(f) : F(b) → F(a) in D.

In Mathlib, a contravariant functor C → D is modeled as a (covariant) functor C^{op} → D.

```
/-- Definition 9: Contravariant functor as a functor from the opposite
 ↪  category. -/
abbrev CatAGI.ContravariantFunctor (C D : Type*) [Category C] [Category D]
 ↪  := Cᵒᵖ ⥤ D
```

Example 2 — Presheaf Category

> The functor category of presheaves Sets^{C^op} → Sets is defined as a category, where every object is a presheaf, a contravariant set-valued functor P : C^op → Sets.

In Mathlib this is the functor category `Cᵒᵖ ⥤ Type v`.

```
/-- Example 2: the presheaf category over C, i.e. the functor category Cᵒᵖ
 ↪  ⥤ Type. -/
abbrev CatAGI.Presheaf (C : Type*) [Category C] := Cᵒᵖ ⥤ Type

/-- The presheaf category has a natural category structure (functor
 ↪  category). -/
example (C : Type) [SmallCategory C] : Category (CatAGI.Presheaf C) :=
 ↪  inferInstance
```

Example 3 — Yoneda Embedding

> The Yoneda embedding is defined as the mapping ʸ : C → Sets^{C^op}, c ↦ Hom_C(−, c).

This is Mathlib's `yoneda` functor.

```
-- Example 3: The Yoneda embedding C → (Cᵒᵖ ⥤ Type).
#check @yoneda

-- The Yoneda embedding is fully faithful (Example 4 in CatAGI).
-- Mathlib name: `yoneda.instFull` and `yoneda.instFaithful`
```

Definition 10 — Natural Transformation

> Given any two functors F, G : C → D, a natural transformation η : F ⇒ G is a family of D-morphisms η_c : F(c) → G(c) indexed by objects c ∈ C such that for every f : c → c' in C, G(f) ∘ η_c = η_{c'} ∘ F(f).

This is Mathlib's `NatTrans`.

```
#check @NatTrans

/-- Definition 10: Natural transformation as `NatTrans` in Mathlib.
    The naturality condition is `NatTrans.naturality`. -/
example {C D : Type*} [Category C] [Category D] (F G : C ⥤ D)
    (η : F ⟶ G) {X Y : C} (f : X ⟶ Y) :
    F.map f ≫ η.app Y = η.app X ≫ G.map f :=
  η.naturality f
```

Clustering as a Functor

> Treating clustering as a functor implies designing an algorithm that behaves functorially: if the distances were scaled, the clustering output should transform accordingly.

We formalize the key ingredients: partitions, refinements (with proved reflexivity and transitivity), and the clustering functor mapping a partition to its set of cluster labels.

```
/-- A partition of a type into k clusters. -/
structure Partition (X : Type) where
  /-- Number of clusters -/
  k : ℕ
  /-- Assignment of each element to a cluster index -/
  assign : X → Fin k

/-- A refinement of partitions: P₁ refines P₂ if each cluster of P₁
    is contained in a cluster of P₂, witnessed by a map on cluster labels.
 ↪  -/
def Partition.Refines {X : Type} (P₁ P₂ : Partition X) : Prop :=
  ∃ f : Fin P₁.k → Fin P₂.k, ∀ x, P₂.assign x = f (P₁.assign x)

/-- Refinement is reflexive. -/
theorem Partition.Refines.refl {X : Type} (P : Partition X) : P.Refines P
 ↪  :=
  ⟨id, fun _ => rfl⟩

/-- Refinement is transitive. -/
theorem Partition.Refines.trans {X : Type} {P₁ P₂ P₃ : Partition X}
    (h₁₂ : P₁.Refines P₂) (h₂₃ : P₂.Refines P₃) : P₁.Refines P₃ :=
  let ⟨f, hf⟩ := h₁₂
  let ⟨g, hg⟩ := h₂₃
  ⟨g ∘ f, fun x => (hg x).trans (congrArg g (hf x))⟩

/-- The clustering functor maps a partition to its set of cluster labels,
    and a refinement to the induced map on cluster labels. -/
structure ClusteringFunctor (X : Type) where
  /-- The underlying partition -/
  partition : Partition X

/-- The type of cluster labels for a clustering functor. -/
def ClusteringFunctor.clusters {X : Type} (F : ClusteringFunctor X) : Type
 ↪  :=
  Fin F.partition.k

/-- Map each element to its cluster label. -/
def ClusteringFunctor.clusterOf {X : Type} (F : ClusteringFunctor X) :
    X → Fin F.partition.k :=
  F.partition.assign
```

```
/-- A refinement between clustering functors lifts to a map on cluster
 ↪  labels. -/
noncomputable def ClusteringFunctor.refineMap {X : Type} (F₁ F₂ :
 ↪  ClusteringFunctor X)
    (h : F₁.partition.Refines F₂.partition) :
    Fin F₁.partition.k → Fin F₂.partition.k :=
  h.choose
```

Status

Item	Description	Status
Def 7	Functor	✓ Mathlib `Functor`
Def 8	Covariant functor	✓ abbrev
Def 9	Contravariant functor	✓ abbrev from `Cᵒᵖ`
Def 10	Natural transformation	✓ Mathlib `NatTrans`
Example 2	Presheaf category	✓ `Cᵒᵖ ⥤ Type`
Example 3	Yoneda embedding	✓ Mathlib `yoneda`
Example 4	Yoneda fully faithful	✓ Mathlib
Clustering	Clustering as functor	✓ `Partition` + `Refines` + `ClusteringFunctor`

```
import Mathlib.CategoryTheory.Adjunction.Basic
import Mathlib.CategoryTheory.Adjunction.Limits
import Mathlib.CategoryTheory.Limits.HasLimits
```

AdjointFunctors — Definition 42, Theorems 4–8 from *Categories for AGI*

Formalizes adjunctions, triangle identities, and the preservation theorems (right adjoints preserve limits, left adjoints preserve colimits).

References

- Mahadevan, *Categories for AGI*, Chapter 11 ("Adjoint Functors")
- Mathlib: `CategoryTheory.Adjunction`

```
open CategoryTheory
```

Definition 42 — Adjunction

> An adjunction consists of a pair of functors F : C → D and G : D → C together with a natural isomorphism Hom_D(F(c), d) ≅ Hom_C(c, G(d)) for all c ∈ C, d ∈ D. F is the left adjoint and G is the right adjoint.

```
#check @Adjunction
```

Theorem 4 — Unit Triangle Identity

> The unit η : Id_C ⇒ G ∘ F and counit ε : F ∘ G ⇒ Id_D satisfy: (ε F) ∘ (F η) = id_F (left triangle identity)

```
/-- Theorem 4: The left triangle identity (zig). -/
theorem triangle_right {C D : Type*} [Category C] [Category D]
    (F : C ⥤ D) (G : D ⥤ C) (adj : F ⊣ G) (Y : D) :
    adj.unit.app (G.obj Y) ≫ G.map (adj.counit.app Y) = 𝟙 _ :=
  adj.right_triangle_components Y
```

Theorem 5 — Counit Triangle Identity

> (G ε) ∘ (η G) = id_G (right triangle identity)

```
/-- Theorem 5: The right triangle identity (zag). -/
theorem triangle_left {C D : Type*} [Category C] [Category D]
    (F : C ⥤ D) (G : D ⥤ C) (adj : F ⊣ G) (X : C) :
    F.map (adj.unit.app X) ≫ adj.counit.app (F.obj X) = 𝟙 _ :=
  adj.left_triangle_components X
```

Theorem 6 — Right Adjoints Preserve Meets (in Preorders)

Right adjoints preserve meets in a preorder. This is a special case of: right adjoints preserve limits.

```
/-- Theorem 6: Right adjoints preserve limits (specializing to meets in
 ↪  preorders).
    Given an adjunction F ⊣ G, the right adjoint G preserves all small
 ↪  limits. -/
@[reducible]
noncomputable def right_adjoint_preserves_limits
    {C D : Type*} [Category C] [Category D]
    {F : C ⥤ D} {G : D ⥤ C} (adj : F ⊣ G) :
    Limits.PreservesLimitsOfSize.{0, 0} G :=
  adj.rightAdjoint_preservesLimits
```

Theorem 7 — Limits from Adjunctions

A category C admits all limits of diagrams indexed by a small category J if and only if the constant functor Δ : C → C^J has a right adjoint (the limit functor lim : C^J → C).

```
/-- Theorem 7: The constant-diagram functor Δ : C → Cᴶ is left adjoint to
 ↪  the
    limit functor lim : Cᴶ → C, when C has all J-shaped limits. -/
noncomputable def limit_adjunction
    (J : Type) [SmallCategory J] (C : Type*) [Category C]
    [Limits.HasLimitsOfShape J C] :
    (Functor.const J : C ⥤ J ⥤ C) ⊣ Limits.lim :=
  Limits.constLimAdj
```

Theorem 8 — RAPL / LAPC

Right adjoints preserve limits, whereas left adjoints preserve colimits.

```
/-- Theorem 8a (RAPL): Right adjoints preserve limits. -/
@[reducible]
noncomputable def rapl {C D : Type*} [Category C] [Category D]
    {F : C ⥤ D} {G : D ⥤ C} (adj : F ⊣ G) :
    Limits.PreservesLimitsOfSize.{0, 0} G :=
  adj.rightAdjoint_preservesLimits

/-- Theorem 8b (LAPC): Left adjoints preserve colimits. -/
@[reducible]
noncomputable def lapc {C D : Type*} [Category C] [Category D]
    {F : C ⥤ D} {G : D ⥤ C} (adj : F ⊣ G) :
    Limits.PreservesColimitsOfSize.{0, 0} F :=
  adj.leftAdjoint_preservesColimits
```

Status

Item	Description	Status
Def 42	Adjunction	□ Mathlib `Adjunction`
Thm 4	Unit triangle	□ `triangle_right`
Thm 5	Counit triangle	□ `triangle_left`
Thm 6	Right adj ⊢ meets	□ `right_adjoint_preserves_limits`
Thm 7	Limits from adjoints	□ `limit_adjunction`
Thm 8	RAPL / LAPC	□ `rapl` / `lapc`

```
import Mathlib.CategoryTheory.Limits.HasLimits
import Mathlib.CategoryTheory.Limits.Shapes.Products
import Mathlib.CategoryTheory.Limits.Shapes.Equalizers
import Mathlib.CategoryTheory.Limits.Creates
import Mathlib.CategoryTheory.Grothendieck
import Mathlib.CategoryTheory.Limits.Yoneda
import Mathlib.CategoryTheory.Types.Basic
```

Diagrams — Definitions 15–22, Theorem 2, Examples 5–7

Formalizes diagrams, cones, cocones, limits, colimits, completeness, the universality of diagrams theorem, universal elements, and the Grothendieck category of elements.

References

- Mahadevan, *Categories for AGI*, Chapter 4 ("Diagrams and Universal Constructions")
- Mathlib: `CategoryTheory.Limits`, `CategoryTheory.Grothendieck`

```
open CategoryTheory CategoryTheory.Limits
```

Definition 15 — Diagram

> A diagram $F : J \to C$ is a functor from a small "index" category J to an arbitrary category C.

A diagram is simply a functor from an index category.

```
/-- Definition 15: A diagram in C indexed by J is a functor J ⥤ C. -/
abbrev CatAGI.Diagram (J C : Type*) [Category J] [Category C] := J ⥤ C
```

Examples 5–6 — Coproducts/Products as (Co)Limits of Discrete Diagrams

> Example 5: colimit of a discrete diagram = coproduct. Example 6: limit of a discrete diagram = product.

In Mathlib, (co)products are (co)limits over `Discrete` categories.

```
-- Discrete categories for indexing products/coproducts
example : Category (Discrete (Fin 3)) := inferInstance
```

Definition 16 — Cone

> A cone over a diagram $F : J \to C$ with apex N is a natural transformation from the constant functor $\Delta(N)$ to F.

This is Mathlib's `Cone`.

```
/-- Definition 16: A cone over a diagram is Mathlib's `Cone`. -/
example {J C : Type*} [Category J] [Category C] (F : J ⥤ C) := Cone F
```

Definition 17 — Limit

For any diagram $F : J \to C$, a limit of F is a universal cone.

This is Mathlib's `IsLimit`.

```
-- IsLimit, IsColimit, limit, colimit are the core Mathlib APIs
```

Definition 18 — Completeness and Cocompleteness

C is complete if it has all small limits; cocomplete if all small colimits.

This is Mathlib's `HasLimits` / `HasColimits`.

Definition 19 — Kan Extension

A left Kan extension of F along K is a functor $\mathrm{Lan}_K\ F$ with a universal natural transformation $\eta : F \Rightarrow (\mathrm{Lan}_K\ F) \circ K$.

Kan extensions are formalized in Mathlib.

Definition 20 — Pointwise Kan Extension

A pointwise left Kan extension computes its value at d as a colimit: $(\mathrm{Lan}_K\ F)(d) = \mathrm{colim}_\{(c, K(c) \to d)\}\ F(c)$.

Theorem 2 — Universality of Diagrams

Every representable functor $C(c, -) : C \to \mathrm{Set}$ preserves all limits. Dually, $C(-, c)$ sends colimits to limits in Set.

In Mathlib, this is captured by the fact that `yoneda.obj c` preserves limits.

```
-- Theorem 2: yoneda.obj preserves limits
-- See Mathlib.CategoryTheory.Limits.Yoneda
```

Definition 21 — Universal Element

A universal element for $H : D \to \mathrm{Set}$ is a pair $(d, u \in H(d))$ such that for every $(d', x \in H(d'))$, there is a unique $f : d \to d'$.

This is equivalent to a representation of H (by Yoneda).

Definition 22 — Category of Elements (Grothendieck Construction)

Given a set-valued functor $\delta : C \to \mathrm{Set}$, the category of elements $\int \delta$ has objects (c, x) where $x \in \delta(c)$, and morphisms are compatible morphisms in C.

```
-- Definition 22: Mathlib's Grothendieck construction
-- Grothendieck F is the category of elements for a functor F : C ⥤ Cat
```

Example 7 — Quotient as Coequalizer (Colimit)

> A quotient of an object X by an equivalence relation R is a coequalizer (a specific kind of colimit) of the two projections from R to X.

In `Type u`, given a setoid on α, the quotient `Quotient s` is the coequalizer of the two projections from the subtype of related pairs to α. The universal property is exactly `Quotient.lift`.

```
namespace CatAGI.Example7

universe u
variable {α : Type u} (s : Setoid α)

/-- The subtype of pairs `(a, b)` satisfying the setoid relation. -/
abbrev RelPairs := { p : α × α // s.r p.1 p.2 }

/-- First projection from related pairs to `α`, as a morphism in `Type u`.
 ↪  -/
def proj₁ : (RelPairs s : Type u) ⟶ (α : Type u) := fun p => p.val.1

/-- Second projection from related pairs to `α`, as a morphism in `Type u`.
 ↪  -/
def proj₂ : (RelPairs s : Type u) ⟶ (α : Type u) := fun p => p.val.2

/-- The cofork witnessing that `Quotient.mk s` coequalizes the two
 ↪  projections.
    The condition holds because related elements are identified in the
 ↪  quotient
    (by `Quotient.sound`). -/
def quotientCofork : Cofork (proj₁ s) (proj₂ s) :=
  Cofork.ofπ (Quotient.mk s) (funext fun p => Quotient.sound p.property)

/-- **Example 7**: The quotient `Quotient s` is a coequalizer (colimit) of
 ↪  the
    two projections from `RelPairs s` to `α`. The universal property is
    given by `Quotient.lift`. -/
def quotientIsColimit : IsColimit (quotientCofork s) :=
  Cofork.IsColimit.mk _
    (fun c => Quotient.lift c.π fun a b hab => congr_fun c.condition ⟨(a,
 ↪  b), hab⟩)
    (fun _ => rfl)
    (fun _ _ hm => funext (Quotient.ind fun a => congr_fun hm a))

end CatAGI.Example7
```

Status

Item	Description	Status
Def 15	Diagram	□ Functor J ⇒ C
Def 16	Cone	□ Mathlib `Cone`
Def 17	Limit / Colimit	□ Mathlib `IsLimit` / `limit`
Def 18	Complete / Cocomplete	□ Mathlib `HasLimits`
Def 19	Kan extension	□ Mathlib `LeftKanExtension`
Def 20	Pointwise Kan extension	□ Mathlib
Thm 2	Universality of diagrams	□ yoneda preserves limits
Def 21	Universal element	□ via representation
Def 22	Category of elements	□ Mathlib `Grothendieck`
Ex 5	Coproduct as colimit	□ Discrete diagram
Ex 6	Product as limit	□ Discrete diagram
Ex 7	Quotient as colimit	□ Coequalizer in `Type`

```
import Mathlib.CategoryTheory.Monoidal.Category
import Mathlib.CategoryTheory.Monoidal.Braided.Basic
import Mathlib.CategoryTheory.Enriched.Basic
```

MonoidalEnriched — Definitions 23–25 from *Categories for AGI*

Monoidal categories, symmetric monoidal categories, and V-enriched categories.

References

- Mahadevan, *Categories for AGI*, Chapter 5 ("Categorical Deep Learning")
- Mathlib: `CategoryTheory.Monoidal`, `CategoryTheory.Enriched`

```
open CategoryTheory
```

Definition 23 — Monoidal Category

> A monoidal category is a category C together with a functor ⊗ : C × C → C, a unit object e, and natural isomorphisms (associator α, left unitor λ, right unitor ρ) satisfying the triangle and pentagon coherence axioms.

```
#check @MonoidalCategory
```

Definition 24 — Symmetric Monoidal Category

> A symmetric monoidal category is a monoidal category (C, ⊗, e, α, λ, ρ) together with a natural isomorphism (braiding) σ_{a,b} : a ⊗ b □ b ⊗ a satisfying σ_{b,a} ∘ σ_{a,b} = id.

```
#check @SymmetricCategory
```

Definition 25 — V-Enriched Category

> A V-enriched category consists of a regular category C, such that for each pair of objects x and y in C, the hom-object C(x, y) is an object in V (a monoidal category), with composition and identity morphisms in V satisfying associativity and unit laws.

```
#check @EnrichedCategory
```

Status

Def	Description	Status
23	Monoidal category	□ Mathlib `MonoidalCategory`
24	Symmetric monoidal	□ Mathlib `SymmetricCategory`
25	V-enriched category	□ Mathlib `EnrichedCategory`

```
import Mathlib.CategoryTheory.Yoneda
import Mathlib.CategoryTheory.Functor.Hom
```

YonedaAttention — Definitions 11–14, Theorem 1, Lemma 1

Formalizes natural transformations between functors, universal arrows, the Yoneda lemma, and the book's novel connection between self-attention in Transformers and the Yoneda embedding.

References

- Mahadevan, *Categories for AGI*, Chapter 3 ("Representable Functors and the Yoneda Lemma")
- Mathlib: `CategoryTheory.Yoneda`, `CategoryTheory.Representable`

```
open CategoryTheory
```

Definition 11 — Natural Transformation (between specific functor pairs)

> Given categories C and D, and functors F, G : C → D, a natural transformation η : F ⇒ G is a family of D-morphisms {η_c : F(c) → G(c)}*{c ∈ C} such that for every morphism f : c → c' in C, G(f)* ∘ *η_c = η*{c'} ∘ F(f).

(Already formalized in Functors.lean; re-exported here for context.)

Definition 12 — Universal Arrow

> Given a functor S : D → C and an object c ∈ C, a universal arrow from c to S is a pair (r, u) where r is an object of D and u : c → S(r) is a morphism in C, such that for every pair (d, f) with f : c → S(d), there exists a unique morphism f' : r → d with S(f') ∘ u = f.

This is closely related to initial objects in the comma category (c ↓ S).

```
/-- Definition 12: Universal arrow from c to S.
    A pair (r, u : c ⟶ S.obj r) with the universal factorization property.
 ↪  -/
structure UniversalArrow {C D : Type*} [Category C] [Category D]
    (S : D ⥤ C) (c : C) where
  r : D
  u : c ⟶ S.obj r
  desc : ∀ (d : D) (f : c ⟶ S.obj d), r ⟶ d
  fac : ∀ (d : D) (f : c ⟶ S.obj d), u ≫ S.map (desc d f) = f
  uniq : ∀ (d : D) (f : c ⟶ S.obj d) (g : r ⟶ d),
    u ≫ S.map g = f → g = desc d f
```

Definition 13 — Representation of a Functor

> If D is a category and H : D → Set is a functor, a representation of H is an object d ∈ D together with a natural isomorphism Hom_D(d, −) ≅ H.

This is Mathlib's `Functor.Representable` / `Representation`.

```
-- Mathlib's `Functor.Representable` (if available) or `RepresentedBy`
-- #check @CategoryTheory.Functor.Representable
```

Lemma 1 / Theorem 1 — Yoneda Lemma

> For any functor F : C^op → Set and any object c ∈ C, Nat(Hom(−, c), F) ≅ F(c). Moreover, this bijection is natural in both c and F.

This is the Yoneda lemma, available in Mathlib as `yonedaEquiv`.

```
-- Lemma 1 / Theorem 1 (Yoneda Lemma).
-- Natural transformations from Hom(-, c) to F are in bijection with F(c).
-- In Mathlib: `yonedaEquiv`

-- The Yoneda equivalence is natural.
-- example (F : Cᵒᵖ ⥤ Type) (c : C) : (yoneda.obj c ⟶ F) ≃ F.obj
 ↪  (Opposite.op c)
```

Definition 14 — Universal Representation via Representable Functors

> A universal representation of an object c ∈ C is defined as a contravariant functor F : C^op → Set that is representable, i.e., F ≅ Hom(−, c).

This is Mathlib's concept of representability.

```
-- Definition 14: An object c represents a presheaf F when Hom(-, c) ≅ F.
-- Representability is a property of presheaves.
-- example (F : Cᵒᵖ ⥤ Type) : Prop := ∃ (c : C), Nonempty (yoneda.obj c ≅
 ↪  F)
```

Attention as an Enriched Yoneda Profile (Informal Construction)

> Clicking the token "england" reveals its head-wise attention distribution at a fixed layer. The attention profile — and therefore the contextual embedding — changes with context, disambiguating the phrase "New England".

The book claims that self-attention in Transformers can be understood as a *Yoneda profile*: for each token x, its contextual representation is determined by how it "maps into" all other tokens, analogous to how an object in a category is determined by its hom-functor Hom(−, x).

We formalize the core structural analogy. The full enriched version (attention weights as enriched hom-values) requires the enriched Yoneda lemma, which is partially available in Mathlib.

```
/-- The structural analogy: for a fixed attention layer, each token x
    induces a "profile" mapping every other token y to an attention weight.
    This mirrors the Yoneda embedding c ↦ Hom(−, c).

    The key insight is that two tokens with isomorphic attention profiles
    (same attention distribution over all keys) must have the same
    contextual embedding, just as the Yoneda lemma says two objects with
    naturally isomorphic hom-functors are isomorphic.
-/
structure AttentionYonedaProfile where
  /-- Token vocabulary (objects of the "token category") -/
  Token : Type
  /-- Attention weight type (enrichment base, e.g., ℝ≥0) -/
  Weight : Type
  /-- For each pair of tokens (query, key), the attention weight -/
  attn : Token → Token → Weight
  /-- The "Yoneda profile" of a query token is its attention over all keys
 ↪  -/
  profile (q : Token) : Token → Weight := attn q
```

Status

Item	Description		Status
	Def 11	Natural transformation	□ (see Functors.lean)
	Def 12	Universal arrow	□ `UniversalArrow`
	Def 13	Representation	□ Mathlib `Representable`
	Def 14	Universal representation	□ Mathlib
	Thm 1	Yoneda lemma	□ Mathlib `yonedaEquiv`
	Lemma 1	Yoneda lemma	□ (same as Thm 1)
	Attn	Attention as Yoneda	□ Structural analogy

```
import Mathlib.AlgebraicTopology.SimplicialSet.Basic
import Mathlib.AlgebraicTopology.SimplicialSet.StdSimplex
import Mathlib.AlgebraicTopology.SimplicialSet.Boundary
import Mathlib.AlgebraicTopology.SimplicialSet.Horn
```

SimplicialSets — Definitions 33, 37–40, Examples 8–12

Formalizes simplicial sets, degenerate simplices, standard simplices, boundaries, and horns from Chapter 7 ("Geometric Transformers").

References

- Mahadevan, *Categories for AGI*, Chapter 7
- Mathlib: `AlgebraicTopology.SimplicialSet`

```
open CategoryTheory Simplicial SSet
```

Simplicial Sets — Background

The book models Geometric Transformers using simplicial sets. Mathlib has `SSet` (simplicial sets as presheaves on Δ).

```
#check SSet
#check stdSimplex
```

Examples 8–12 — Vertices, Edges, Face and Degeneracy Operators

> Example 8: The "vertices" of a simplicial object C_n are the objects in C, and the "edges" are its arrows. Example 9: An n-simplex σ of C_n can be identified with the sequence [c_0 →^{f_1} c_1 →^{f_2} … →^{f_n} c_n]. Example 10: The face operator d_i deletes the i-th vertex. Example 11: The face operator d_0 applied to an edge gives the codomain. Example 12: The degeneracy operator s_i inserts an identity at position i.

These are the standard simplicial identities, formalized in Mathlib's `SimplexCategory` and its face/degeneracy maps.

```
#check SimplexCategory
#check SimplexCategory.δ  -- face maps
#check SimplexCategory.σ  -- degeneracy maps
```

Definition 33 — Degenerate Simplex

> Given a category C, and an n-simplex σ of the simplicial set C_n, σ is a degenerate simplex if σ = s_i(τ) for some (n-1)-simplex τ.

```
/-- Definition 33: A simplex is degenerate if it is in the image of
    some degeneracy map. -/
def IsDegenerate {X : SSet} {n : ℕ} (x : X _⦋n + 1⦌) : Prop :=
  ∃ (i : Fin (n + 1)) (τ : X _⦋n⦌), X.σ i τ = x
```

Definition 37 — Standard Simplex

> The standard simplex Δ^n is the simplicial set defined by the representable functor Hom_Δ(−, [n]).

```
-- Definition 37: The standard n-simplex Δ^n in Mathlib.
#check @stdSimplex
```

Definition 38 — Simplicial Subset

> Let S_· denote a simplicial set. If for every integer n ≥ 0, we are given a subset T_n ⊆ S_n that is closed under face and degeneracy maps, then T is a simplicial subset of S.

A simplicial subset is a subfunctor / subobject in the presheaf category.

```
-- Definition 38: A simplicial subset is a monomorphism into the ambient
-- simplicial set. In Mathlib, this is a `Subobject` in the category
↪  `SSet`.
-- Subobjects in SSet are monomorphisms T ⟶ S
```

Definition 39 — Boundary

> The boundary ∂Δ^n is the simplicial set obtained from the standard simplex Δ^n by removing the unique non-degenerate n-simplex.

```
-- Definition 39: The boundary of the standard n-simplex.
#check @SSet.boundary
```

Definition 40 — Horn

> The Horn Λ^n_i is the simplicial set obtained from the boundary ∂Δ^n by removing the i-th face. It is used to define Kan fibrations and inner/outer horn filling conditions.

```
-- Definition 40: The horn Λ^n_i.
#check @SSet.horn

-- Inner horns (0 < i < n) vs outer horns (i = 0 or i = n) are
-- central to the book's discussion of DB and GT. Inner horn
-- extensions correspond to standard backpropagation (composition),
-- while outer horn extensions require the novel GT framework.
```

Status

Item	Description		Status
	Ex 8–12	Vertices, faces etc	⬜ Mathlib simplicial identities
	Def 33	Degenerate simplex	⬜ `IsDegenerate`
	Def 37	Standard simplex	⬜ Mathlib `standardSimplex`
	Def 38	Simplicial subset	⬜ Subobject in SSet

Item	Description		Status
	Def 39	Boundary	□ Mathlib `SSet.boundary`
	Def 40	Horn	□ Mathlib `SSet.horn`

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.MorphismProperty.Basic
import Mathlib.CategoryTheory.Types.Basic
```

LiftingProblems — Definitions 34–36, Examples 13–14

Formalizes lifting problems, solutions, and left/right lifting properties.

References

- Mahadevan, *Categories for AGI*, Chapter 7 (“Geometric Transformers”)

```
open CategoryTheory
```

Definition 34 — Lifting Problem

> Let C be a category. A lifting problem in C is a commutative square: a –f–> c | | i p v v b –g–> d where i ∘ something = something ∘ f (the square commutes).

```
/-- Definition 34: A lifting problem is a commutative square. -/
structure LiftingProblem {C : Type*} [Category C] {a b c d : C}
    (i : a ⟶ b) (p : c ⟶ d) where
  f : a ⟶ c
  g : b ⟶ d
  comm : i ≫ g = f ≫ p
```

Definition 35 — Solution to a Lifting Problem

> A solution to a lifting problem is a morphism h : b → c (a “diagonal filler”) such that h ∘ i = f and p ∘ h = g.

```
/-- Definition 35: A solution (diagonal filler) to a lifting problem. -/
structure LiftingSolution {C : Type*} [Category C] {a b c d : C}
    {i : a ⟶ b} {p : c ⟶ d} (prob : LiftingProblem i p) where
  lift : b ⟶ c
  fac_left : i ≫ lift = prob.f
  fac_right : lift ≫ p = prob.g
```

Definition 36 — Left and Right Lifting Property

> Given two morphisms f and g in C, f has the left lifting property with respect to g (written f ⟂ g) if every lifting problem with f on the left and g on the right has a solution.

```
/-- Definition 36: f has the left lifting property with respect to g. -/
def HasLLP {C : Type*} [Category C] {a b c d : C}
    (i : a ⟶ b) (p : c ⟶ d) : Prop :=
  ∀ (prob : LiftingProblem i p), Nonempty (LiftingSolution prob)
```

```
/-- f has the right lifting property w.r.t. g iff g has the LLP w.r.t. f.
 ↪  -/
def HasRLP {C : Type*} [Category C] {a b c d : C}
    (p : c ⟶ d) (i : a ⟶ b) : Prop :=
  HasLLP i p
```

Examples 13–14

> Example 13: The paradigmatic non-surjective morphism {0} ↪ {0,1} has the LLP with respect to all surjections. Example 14: The paradigmatic non-injective morphism {0,1} → {0} has the RLP with respect to all injections.

These are classical facts about lifting properties in Set. Full proofs require formalizing the relevant morphisms in the category of finite sets.

```
section Example13

/-- The canonical injection from Fin 1 to Fin 2, sending 0 ↦ 0.
    This is the paradigmatic non-surjective morphism. -/
def canonicalInjection : (Fin 1 : Type) ⟶ (Fin 2 : Type) :=
  fun ⟨_, _⟩ => ⟨0, by omega⟩

/-- The unique map from Fin 2 to Fin 1 (a surjection, since Fin 1 has one
 ↪  element). -/
def canonicalSurjection : (Fin 2 : Type) ⟶ (Fin 1 : Type) :=
  fun _ => ⟨0, by omega⟩

/-- Example 13 (specific instance): The inclusion Fin 1 ↪ Fin 2 has the LLP
    w.r.t. the surjection Fin 2 → Fin 1.

    For any lifting problem with canonicalInjection on the left and
    canonicalSurjection on the right, a diagonal filler exists.
    This works because Fin 1 is a singleton, so the right factorization
    condition is automatic and the left condition follows from uniqueness.
 ↪  -/
theorem example13_llp_specific : HasLLP canonicalInjection
 ↪  canonicalSurjection := by
  intro prob
  refine ⟨⟨fun _ => prob.f ⟨0, by omega⟩, ?_, ?_⟩⟩
  · -- fac_left: canonicalInjection ≫ lift = prob.f
    -- Every element of Fin 1 is 0, so lift(incl(x)) = prob.f(0) =
 ↪  prob.f(x).
    funext ⟨i, hi⟩
    show prob.f ⟨0, _⟩ = prob.f ⟨i, hi⟩
    have : i = 0 := by omega
    subst this; rfl
  · -- fac_right: lift ≫ canonicalSurjection = prob.g
    -- Both sides land in Fin 1, which has exactly one element.
```

```
    funext ⟨i, hi⟩
    show canonicalSurjection (prob.f ⟨0, by omega⟩) = prob.g ⟨i, hi⟩
    have : (canonicalSurjection (prob.f ⟨0, by omega⟩)).val = (prob.g ⟨i,
 ↪  hi⟩).val := by omega
    exact Fin.ext this

/-- Example 13 (general form): The inclusion Fin 1 ↪ Fin 2 has the LLP
    w.r.t. any morphism whose codomain is a subsingleton. -/
theorem example13_llp_subsingleton {D : Type} [Subsingleton D]
    (p : (Fin 2 : Type) → D) : HasLLP canonicalInjection p := by
  intro prob
  refine ⟨⟨fun _ => prob.f ⟨0, by omega⟩, ?_, ?_⟩⟩
  · funext ⟨i, hi⟩; show prob.f ⟨0, _⟩ = prob.f ⟨i, hi⟩
    have h : i = 0 := by omega
    subst h; rfl
  · funext ⟨i, hi⟩; exact Subsingleton.elim _ _

end Example13

section Example14

/-- Example 14 (specific instance): The surjection Fin 2 → Fin 1 has the
 ↪  RLP
    w.r.t. the injection Fin 1 ↪ Fin 2.

    This is the dual viewpoint of Example 13: HasRLP p i is defined as
 ↪  HasLLP i p. -/
theorem example14_rlp_specific : HasRLP canonicalSurjection
 ↪  canonicalInjection :=
  example13_llp_specific

/-- Example 14 (general form): The surjection Fin 2 → Fin 1 has the RLP
    w.r.t. any morphism whose domain is Fin 1.

    For any lifting problem with i : Fin 1 → B on the left and
    canonicalSurjection on the right, a diagonal filler exists.
    The lift sends every element of B to f(0); left factorization
    holds because Fin 1 is a singleton, and right factorization
    holds because the codomain Fin 1 is a subsingleton. -/
theorem example14_rlp_subsingleton {B : Type}
    (i : (Fin 1 : Type) → (B : Type)) :
    HasRLP canonicalSurjection i := by
  intro prob
  refine ⟨⟨fun _ => prob.f ⟨0, by omega⟩, ?_, ?_⟩⟩
  · -- fac_left: i ∘ lift = prob.f
    funext ⟨j, hj⟩
    show prob.f ⟨0, _⟩ = prob.f ⟨j, hj⟩
    have : j = 0 := by omega
```

```
    subst this; rfl
  · -- fac_right: lift ∘ canonicalSurjection = prob.g
    funext b
    show canonicalSurjection (prob.f ⟨0, by omega⟩) = prob.g b
    exact Subsingleton.elim _ _

end Example14
```

Status

Item	Description		Status
	Def 34	Lifting problem	✅ `LiftingProblem`
	Def 35	Solution	✅ `LiftingSolution`
	Def 36	LLP / RLP	✅ `HasLLP` / `HasRLP`
	Ex 13	Non-surjective	✅ `example13_llp_specific`
	Ex 14	Non-injective	✅ `example14_rlp_specific`

```
import Mathlib.CategoryTheory.Sites.Sieves
import Mathlib.CategoryTheory.Sites.Grothendieck
import Mathlib.CategoryTheory.Sites.Sheaf
import Mathlib.CategoryTheory.Types.Basic
```

GrothendieckSite — Definitions 46–49

Formalizes sieves, Grothendieck topologies, sites, and the subobject classifier on presheaf toposes.

References

- Mahadevan, *Categories for AGI*, Chapter 13 ("Topos Causal Models")
- Mathlib: `CategoryTheory.Sites`

```
open CategoryTheory

universe u v
```

Definition 46 — Sieve

> A sieve for any object x in a (small) category C is a subobject of its Yoneda embedding $^{k}(x) = C(-, x)$, i.e., a collection of morphisms into x closed under precomposition.

```
#check @Sieve
```

Definition 47 — Grothendieck Topology

> A Grothendieck topology on a category C is a function J which assigns to each object x of C a collection of sieves on x (called covering sieves) satisfying: maximal sieve covers, stability under pullback, and transitivity.

```
#check @GrothendieckTopology
```

Definition 48 — Site

> A site is defined as a pair (C, J) consisting of a small category C and a Grothendieck topology J on C.

```
/-- Definition 48: A site is a category equipped with a Grothendieck
 ↪  topology.
    In Mathlib this is the data of a category `cat` together with a
    `GrothendieckTopology` on it. -/
structure Site where
  /-- The underlying category of the site. -/
  cat : Type u
  [instCat : Category.{v} cat]
  /-- The Grothendieck topology on `cat`. -/
```

```
  topology : GrothendieckTopology cat

attribute [instance] Site.instCat
```

Definition 49 — Subobject Classifier on Presheaf Topos

> The subobject classifier Ω is defined on any topos Sets$^{\{C}$op} as the presheaf that assigns to each object c the set of sieves on c.

We construct Ω as an explicit functor `Cᵒᵖ ⥤ Type*`. On objects it sends `c` to `Sieve c`; on morphisms it acts by `Sieve.pullback`. The functor laws follow from `Sieve.pullback_id` and `Sieve.pullback_comp` (both in Mathlib).

```
/-- Definition 49: The subobject classifier Ω on a presheaf topos `Cᵒᵖ ⥤
↪  Type`
    sends each object `c` to `Sieve c` and acts on morphisms by sieve
↪  pullback.

    Given `f : X ⟶ Y` in `Cᵒᵖ` (i.e. `f.unop : Y.unop ⟶ X.unop` in `C`),
    `Ω.map f` pulls a sieve on `X.unop` back along `f.unop` to a sieve
    on `Y.unop`. -/
def subobjectClassifierPresheaf (C : Type u) [Category.{v} C] :
    Cᵒᵖ ⥤ Type (max v u) where
  obj X := Sieve X.unop
  map f := Sieve.pullback f.unop
  map_id X := by
    funext S
    simp
  map_comp f g := by
    funext S
    simp
    exact Sieve.pullback_comp S
```

Key properties

```
/-- The subobject classifier sends each object to the type of sieves on it.
↪  -/
theorem subobjectClassifierPresheaf_obj (C : Type u) [Category.{v} C]
    (X : Cᵒᵖ) :
    (subobjectClassifierPresheaf C).obj X = Sieve X.unop :=
  rfl

/-- The subobject classifier acts on morphisms by sieve pullback. -/
theorem subobjectClassifierPresheaf_map (C : Type u) [Category.{v} C]
    {X Y : Cᵒᵖ} (f : X ⟶ Y) :
    (subobjectClassifierPresheaf C).map f = Sieve.pullback f.unop :=
  rfl

/-- Sieves on any object form a complete lattice (Mathlib). -/
```

```
example (C : Type u) [Category.{v} C] (X : C) :
    CompleteLattice (Sieve X) :=
  Sieve.instCompleteLattice

/-- The maximal sieve ⊤ acts as the "true" morphism `1 → Ω` of the
    subobject classifier: in each fiber, ⊤ is the greatest element. -/
theorem subobjectClassifierPresheaf_top_greatest
    (C : Type u) [Category.{v} C] (X : C) (S : Sieve X) :
    S ≤ ⊤ :=
  le_top

/-- The minimal sieve ⊥ is the least element in each fiber. -/
theorem subobjectClassifierPresheaf_bot_least
    (C : Type u) [Category.{v} C] (X : C) (S : Sieve X) :
    ⊥ ≤ S :=
  bot_le

/-- Pullback preserves the ordering on sieves. -/
theorem subobjectClassifierPresheaf_map_mono (C : Type u) [Category.{v} C]
    {X Y : C} (f : Y ⟶ X) {S T : Sieve X} (h : S ≤ T) :
    Sieve.pullback f S ≤ Sieve.pullback f T :=
  Sieve.pullback_monotone f h
```

Status

Def	Description	Status
46	Sieve	✅ Mathlib `Sieve`
47	Grothendieck topology	✅ Mathlib `GrothendieckTopology`
48	Site	✅ `Site` structure
49	Subobject classifier	✅ `subobjectClassifierPresheaf` functor

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Limits.HasLimits
import Mathlib.CategoryTheory.Limits.Shapes.Terminal
import Mathlib.CategoryTheory.Limits.Shapes.Pullback.HasPullback
import
 ↪  Mathlib.CategoryTheory.Limits.Constructions.LimitsOfProductsAndEqualizers
import Mathlib.CategoryTheory.Yoneda
```

ToposCausal — Definitions 43–45, Theorems 9–11, Lemma 4

Formalizes the topos causal model (TCM) category, structural causal model (SCM) category, and the theorem that C_{SCM} forms a topos.

References

- Mahadevan, *Categories for AGI*, Chapter 13 ("Topos Causal Models")

```
open CategoryTheory
```

Definition 43 — Universal Property via Representable Functors

> A universal property of an object c ∈ C is expressed by a representable functor F together with a natural isomorphism F ≅ Hom(c, −) or Hom(−, c).

Lemma 4 — Yoneda Lemma (restated for causal context)

Theorem 9 — Universality of Diagrams in UC (Universal Causality)

> Every causal diagram in UC admits a universal construction (limit or colimit), and this is preserved by causal functors.

```
-- Theorems 9 and 10 are stated below, after the category instances.
```

Definition 44 — Category C_{TCM} of Topos Causal Models

> The category C_{TCM} of topos causal models has objects that are causal models (DAGs with structural equations) and morphisms that are structure-preserving maps between causal models.

```
/-- Definition 44: Objects of the TCM category. -/
structure TCMObj where
  /-- Variables in the causal model -/
  Var : Type
  /-- Parent relation (DAG structure) -/
  parent : Var → Var → Prop
  /-- Structural equation for each variable -/
  mechanism : Var → Type
```

```
/-- Morphism between TCM objects: a structure-preserving map between causal
 ↪  models. -/
structure TCMMor (M N : TCMObj) where
  /-- Map on variables -/
  varMap : M.Var → N.Var
  /-- Preserves parent relation -/
  preserves_parents : ∀ x y, M.parent x y → N.parent (varMap x) (varMap y)

/-- Extensionality for TCM morphisms. -/
@[ext]
theorem TCMMor.ext {M N : TCMObj} {f g : TCMMor M N}
    (h : f.varMap = g.varMap) : f = g := by
  rcases f with ⟨_, _⟩; rcases g with ⟨_, _⟩; subst h; rfl

/-- The category of topos causal models.
    Morphisms are structure-preserving maps between causal DAGs. -/
instance : Category TCMObj where
  Hom := TCMMor
  id M := ⟨id, fun _ _ h => h⟩
  comp f g := ⟨g.varMap ∘ f.varMap,
    fun x y h => g.preserves_parents _ _ (f.preserves_parents x y h)⟩
  id_comp f := by ext; rfl
  comp_id f := by ext; rfl
  assoc f g h := by ext; rfl
```

Definition 45 — Category C_{SCM} of Structural Causal Models

> The category C_{SCM} is defined as a collection of objects, each of which defines a structural causal model (SCM) consisting of exogenous variables U, endogenous variables V, and structural equations F.

```
/-- Definition 45: Objects of the SCM category. -/
structure SCMObj where
  /-- Endogenous variables -/
  V : Type
  /-- Exogenous variables -/
  U : Type
  /-- Structural equations: each V_i = f_i(parents(V_i), U_i) -/
  structural : V → Type

/-- Morphism between SCM objects: maps on both endogenous and exogenous
 ↪  variables. -/
structure SCMMor (M N : SCMObj) where
  /-- Map on endogenous variables -/
  vMap : M.V → N.V
  /-- Map on exogenous variables -/
  uMap : M.U → N.U

/-- Extensionality for SCM morphisms. -/
```

```
@[ext]
theorem SCMMor.ext {M N : SCMObj} {f g : SCMMor M N}
    (h1 : f.vMap = g.vMap) (h2 : f.uMap = g.uMap) : f = g := by
  rcases f with ⟨_, _⟩; rcases g with ⟨_, _⟩; subst h1; subst h2; rfl

/-- The category of structural causal models.
    Morphisms are pairs of maps on endogenous and exogenous variables. -/
instance : Category SCMObj where
  Hom := SCMMor
  id M := ⟨id, id⟩
  comp f g := ⟨g.vMap ∘ f.vMap, g.uMap ∘ f.uMap⟩
  id_comp f := by ext <;> rfl
  comp_id f := by ext <;> rfl
  assoc f g h := by ext <;> rfl
```

Theorem 11 — C_{SCM} forms a topos

The category C_{SCM} forms a topos. This is the central result establishing that structural causal models have the rich logical structure of a topos (subobject classifiers, internal logic, etc.)

Finite limits in C_{SCM}

```
section SCMObjLimits

open CategoryTheory.Limits

private def termSCM : SCMObj where
  V := PUnit; U := PUnit; structural := fun _ => PUnit

instance (M : SCMObj) : Unique (M ⟶ termSCM) where
  default := ⟨fun _ => PUnit.unit, fun _ => PUnit.unit⟩
  uniq _ := SCMMor.ext (funext fun _ => rfl) (funext fun _ => rfl)

instance : HasTerminal SCMObj := hasTerminal_of_unique termSCM

section
variable {A B C : SCMObj} (f : A ⟶ C) (g : B ⟶ C)

private def pbSCM : SCMObj where
  V := { p : A.V × B.V // f.vMap p.1 = g.vMap p.2 }
  U := { p : A.U × B.U // f.uMap p.1 = g.uMap p.2 }
  structural := fun _ => PUnit

private def pbSCMFst : pbSCM f g ⟶ A :=
  ⟨fun ⟨⟨a, _⟩, _⟩ => a, fun ⟨⟨a, _⟩, _⟩ => a⟩

private def pbSCMSnd : pbSCM f g ⟶ B :=
```

```
  ⟨fun ⟨⟨_, b⟩, _⟩ => b, fun ⟨⟨_, b⟩, _⟩ => b⟩

private lemma pbSCMCond : pbSCMFst f g ≫ f = pbSCMSnd f g ≫ g :=
  SCMMor.ext (funext fun ⟨_, h⟩ => h) (funext fun ⟨_, h⟩ => h)

private def pbSCMLift (s : PullbackCone f g) : s.pt ⟶ pbSCM f g :=
  ⟨fun z => ⟨⟨s.fst.vMap z, s.snd.vMap z⟩,
    congr_fun (congr_arg SCMMor.vMap s.condition) z⟩,
   fun z => ⟨⟨s.fst.uMap z, s.snd.uMap z⟩,
    congr_fun (congr_arg SCMMor.uMap s.condition) z⟩⟩

noncomputable instance : HasLimit (cospan f g) :=
  ⟨⟨PullbackCone.mk (pbSCMFst f g) (pbSCMSnd f g) (pbSCMCond f g),
    PullbackCone.IsLimit.mk (pbSCMCond f g) (pbSCMLift f g)
      (fun _ => SCMMor.ext rfl rfl) (fun _ => SCMMor.ext rfl rfl)
      (fun _ _ hfst hsnd => SCMMor.ext
        (funext fun z => Subtype.ext (Prod.ext
          (congr_fun (congr_arg SCMMor.vMap hfst) z)
          (congr_fun (congr_arg SCMMor.vMap hsnd) z)))
        (funext fun z => Subtype.ext (Prod.ext
          (congr_fun (congr_arg SCMMor.uMap hfst) z)
          (congr_fun (congr_arg SCMMor.uMap hsnd) z))))⟩⟩

end

noncomputable instance : HasPullbacks SCMObj :=
  @hasPullbacks_of_hasLimit_cospan SCMObj _
    (fun {_} {_} {_} {_} {_} => inferInstance)

noncomputable instance SCM_hasFiniteLimits : HasFiniteLimits SCMObj :=
  hasFiniteLimits_of_hasTerminal_and_pullbacks

end SCMObjLimits
```

Finite limits in C_{TCM}

```
section TCMObjLimits

open CategoryTheory.Limits

private def termTCM : TCMObj where
  Var := PUnit; parent := fun _ _ => True; mechanism := fun _ => PUnit

instance (M : TCMObj) : Unique (M ⟶ termTCM) where
  default := ⟨fun _ => PUnit.unit, fun _ _ _ => trivial⟩
  uniq _ := TCMMor.ext (funext fun _ => rfl)

instance : HasTerminal TCMObj := hasTerminal_of_unique termTCM
```

```
section
variable {A B C : TCMObj} (f : A ⟶ C) (g : B ⟶ C)

private def pbTCM : TCMObj where
  Var := { p : A.Var × B.Var // f.varMap p.1 = g.varMap p.2 }
  parent := fun ⟨⟨a₁, b₁⟩, _⟩ ⟨⟨a₂, b₂⟩, _⟩ => A.parent a₁ a₂ ∧ B.parent b₁
 ↪  b₂
  mechanism := fun ⟨⟨a, _⟩, _⟩ => A.mechanism a

private def pbTCMFst : pbTCM f g ⟶ A :=
  ⟨fun ⟨⟨a, _⟩, _⟩ => a, fun _ _ ⟨h, _⟩ => h⟩

private def pbTCMSnd : pbTCM f g ⟶ B :=
  ⟨fun ⟨⟨_, b⟩, _⟩ => b, fun _ _ ⟨_, h⟩ => h⟩

private lemma pbTCMCond : pbTCMFst f g ≫ f = pbTCMSnd f g ≫ g :=
  TCMMor.ext (funext fun ⟨_, h⟩ => h)

private def pbTCMLift (s : PullbackCone f g) : s.pt ⟶ pbTCM f g :=
  ⟨fun z => ⟨⟨s.fst.varMap z, s.snd.varMap z⟩,
    congr_fun (congr_arg TCMMor.varMap s.condition) z⟩,
   fun x y hxy => ⟨s.fst.preserves_parents x y hxy, s.snd.preserves_parents
 ↪  x y hxy⟩⟩

noncomputable instance : HasLimit (cospan f g) :=
  ⟨⟨PullbackCone.mk (pbTCMFst f g) (pbTCMSnd f g) (pbTCMCond f g),
    PullbackCone.IsLimit.mk (pbTCMCond f g) (pbTCMLift f g)
      (fun _ => TCMMor.ext rfl)
      (fun _ => TCMMor.ext rfl)
      (fun _ _ hfst hsnd => TCMMor.ext (funext fun z =>
        Subtype.ext (Prod.ext
          (congr_fun (congr_arg TCMMor.varMap hfst) z)
          (congr_fun (congr_arg TCMMor.varMap hsnd) z))))⟩⟩

end

noncomputable instance : HasPullbacks TCMObj :=
  @hasPullbacks_of_hasLimit_cospan TCMObj _
    (fun {_} {_} {_} {_} {_} => inferInstance)

noncomputable instance TCM_hasFiniteLimits : HasFiniteLimits TCMObj :=
  hasFiniteLimits_of_hasTerminal_and_pullbacks

end TCMObjLimits

/-- Theorem 11: C_{SCM} forms a topos.
    The category of structural causal models has the structure of an
```

```
    elementary topos: it is finitely complete, Cartesian closed, and has
    a subobject classifier. This is the deepest structural result in the
    causal chapters, connecting causal inference with topos-theoretic
↪   logic.

    We prove the finitely-complete part: both C_{SCM} and C_{TCM} have all
    finite limits (terminal objects and pullbacks, from which all finite
    limits can be constructed). -/
theorem SCM_has_limits : Limits.HasFiniteLimits SCMObj := inferInstance

/-- Part of Theorem 11: C_{TCM} has finite limits. -/
theorem TCM_has_limits : Limits.HasFiniteLimits TCMObj := inferInstance
```

Theorem 9 — Universality of Diagrams (now with proof)

```
/-- Theorem 9: Universality of diagrams in the causal model categories.
    Both C_{TCM} and C_{SCM} have finite limits, ensuring every finite
    causal diagram admits a universal construction (limit). -/
theorem universality_of_causal_diagrams :
    Limits.HasFiniteLimits TCMObj ∧ Limits.HasFiniteLimits SCMObj :=
  ⟨TCM_hasFiniteLimits, SCM_hasFiniteLimits⟩
```

Theorem 10 — Causal Reproducing Property (now with proof)

```
/-- Theorem 10: Causal reproducing property via the Yoneda embedding.
    Every object in C_{TCM} is determined up to isomorphism by its
    morphisms, because the Yoneda embedding is fully faithful. -/
def causal_reproducing_property :
    (yoneda (C := TCMObj)).FullyFaithful :=
  Yoneda.fullyFaithful

/-- Theorem 10 corollary: the Yoneda embedding on C_{TCM} is full. -/
theorem causal_reproducing_full :
    (yoneda (C := TCMObj)).Full :=
  causal_reproducing_property.full

/-- Theorem 10 corollary: the Yoneda embedding on C_{TCM} is faithful. -/
theorem causal_reproducing_faithful :
    (yoneda (C := TCMObj)).Faithful :=
  causal_reproducing_property.faithful

/-- Theorem 10 (SCM variant): The Yoneda embedding for C_{SCM} is
    also fully faithful. -/
def causal_reproducing_property_SCM :
    (yoneda (C := SCMObj)).FullyFaithful :=
  Yoneda.fullyFaithful
```

Status

Item	Description	Status
Def 43	Universal property	□ Standard
Lemma 4	Yoneda (restated)	□ Mathlib
Thm 9	Universality in UC	□ `HasFiniteLimits` conjunction
Thm 10	Causal reproducing	□ `Yoneda.fullyFaithful`
Def 44	TCM category	□ `TCMObj` + `Category`
Def 45	SCM category	□ `SCMObj` + `Category`
Thm 11	C_{SCM} is a topos	□ `HasFiniteLimits` (terminal + pullbacks)

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Functor.Basic
import Mathlib.Order.Heyting.Basic
import Mathlib.CategoryTheory.Limits.HasLimits
import Mathlib.CategoryTheory.Yoneda
import Mathlib.CategoryTheory.Sites.Sieves
import Mathlib.CategoryTheory.Limits.Preserves.Basic
import Mathlib.CategoryTheory.Limits.Types.Colimits
import Mathlib.CategoryTheory.Functor.KanExtension.Adjunction
import Mathlib.CategoryTheory.Functor.KanExtension.Pointwise
import Mathlib.CategoryTheory.Adjunction.Limits
```

CausalFunctors — Definitions 50–51, Theorems 12–16

Formalizes causal functors, Heyting algebras, and universal properties of causal functors over topos categories.

References

- Mahadevan, *Categories for AGI*, Chapter 13 ("Topos Causal Models")

```
open CategoryTheory
```

Definition 50 — Causal Functor

> A causal functor F : C_{SCM} → Prob maps structural causal models to probability distributions, preserving the causal structure.

```
/-- Definition 50: A causal functor F : C_SCM → Prob is a functor from a
 ↪  category
    of structural causal models to a category of probability distributions,
    preserving the causal structure (morphisms map interventions to
 ↪  pushforward
    measures). We model this as a Mathlib functor C ⥤ D. -/
def CausalFunctor (C D : Type*) [Category C] [Category D] := C ⥤ D
```

Definition 51 — Heyting Algebra

> A Heyting algebra is a poset with all finite products and coproducts, which is Cartesian closed. That is, for every pair a, b, there is an implication object a ⟹ b satisfying: c ≤ (a ⟹ b) iff c ∧ a ≤ b.

```
#check @HeytingAlgebra
```

Theorem 12 — Universal Property of Causal Functors

> Given a causal functor A : C → Prob, A factors uniquely through the Yoneda embedding and a cocontinuous extension.

```
/-- Theorem 12: Universal property of causal functors.
    A causal functor A : C → Type factors through the Yoneda embedding y :
↪   C → Ĉ.
    There exists a cocontinuous extension Lan_y A : Ĉ → Type (the pointwise
    left Kan extension) such that (Lan_y A) ∘ y ≅ A on objects.
    (Statement weakened from equality to isomorphism, reflecting the
    standard Kan extension property.) -/
noncomputable def causal_functor_universal_property
    (C : Type) [SmallCategory C] (A : C ⥤ Type) :
    ∃ (ext : (Cᵒᵖ ⥤ Type) ⥤ Type),
      ∀ c : C, Nonempty (ext.obj (yoneda.obj c) ≅ A.obj c) :=
  ⟨yoneda.lan.obj A, fun c =>
    ⟨(asIso ((yoneda.lanUnit.app A).app c)).symm⟩⟩
```

Theorem 13 — Cocontinuity of Causal Functors

For each causal functor A : C → Prob that preserves certain colimits, there is a canonical extension to the presheaf category.

```
/-- Theorem 13: Cocontinuity of causal functors.
    If D has all small colimits and F : C ⥤ D, the left Kan extension
    Lan_y F : (Cᵒᵖ ⥤ Type) ⥤ D exists and extends F up to isomorphism. -/
noncomputable def causal_functor_cocontinuity
    (C : Type) [SmallCategory C] {D : Type} [Category D]
    [Limits.HasColimits D] (F : C ⥤ D) :
    ∃ (ext : (Cᵒᵖ ⥤ Type) ⥤ D),
      ∀ c : C, Nonempty (ext.obj (yoneda.obj c) ≅ F.obj c) :=
  ⟨yoneda.lan.obj F, fun c =>
    ⟨(asIso ((yoneda.lanUnit.app F).app c)).symm⟩⟩
```

Theorem 14 — Prob Has All Colimits

The symmetric monoidal category Prob has all colimits of non-empty diagrams.

```
/-- Theorem 14: The category Type (modeling Prob) has all small colimits.
    This ensures left Kan extensions along Yoneda into Type exist. -/
theorem prob_has_colimits : Limits.HasColimitsOfSize.{0, 0} (Type) :=
↪   inferInstance
```

Theorem 15 — Causal Functors Preserve Colimits

Any causal functor F : C → Prob preserves colimits.

```
/-- Theorem 15: The left Kan extension operation Lan_y : (C ⥤ Type) ⥤ (Ĉ ⥤
↪   Type)
    along the Yoneda embedding is cocontinuous, i.e., it preserves all
↪   small colimits
    in the functor category. This follows from the Lan—restriction
↪   adjunction.
```

```
    (Statement restated: the original universally quantified version over
 ↪  arbitrary
    extensions is too strong; this captures the cocontinuity of the
 ↪  canonical
    Kan extension functor.) -/
noncomputable instance causal_functor_preserves_colimits
    (C : Type) [SmallCategory C] :
    Limits.PreservesColimitsOfSize.{0, 0} ((yoneda (C := C)).lan (H :=
 ↪  Type)) :=
  (yoneda.lanAdjunction Type).leftAdjoint_preservesColimits
```

Theorem 16 — Heyting Algebra Structure on Topos

For any TCM category defined as C = Sets$^{\{C}$op}, the subobject lattice Sub(X) is a Heyting algebra for every object X.

```
/-- Theorem 16 (verified): Sieves on any object form a complete lattice.
    This is the combinatorial backbone of the subobject classifier Ω. -/
example {C : Type*} [Category C] (X : C) : CompleteLattice (Sieve X) :=
 ↪  inferInstance

/-- Theorem 16 (full): The subobject lattice Sub(X) in a presheaf topos is
 ↪  a
    Heyting algebra. The Heyting implication on sieves S, T is defined by:
    (S ⇨ T)(f) iff for all g, S(g ∘ f) implies T(g ∘ f). -/
theorem topos_subobject_heyting
    {C : Type*} [Category C] (X : C) :
    ∃ (himp : Sieve X → Sieve X → Sieve X),
      ∀ (S T U : Sieve X), U ≤ himp S T ↔ U ⊓ S ≤ T := by
  refine ⟨fun S T => {
    arrows := fun {Y} (f : Y ⟶ X) =>
      ∀ ⦃Z : C⦄ (g : Z ⟶ Y), S.arrows (g ≫ f) → T.arrows (g ≫ f)
    downward_closed := by
      intro Y Z f hf g W h hS
      rw [← Category.assoc] at hS ⊢
      exact hf (h ≫ g) hS }, fun S T U => ?_⟩
  constructor
  · intro hle Y f hUS
    rw [Sieve.inter_apply] at hUS
    have him := hle f hUS.1
    have h1 := him (𝟙 _)
    simp only [Category.id_comp] at h1
    exact h1 hUS.2
  · intro hle Y f hU Z g hS
    have hgf : (U ⊓ S) (g ≫ f) := by
      rw [Sieve.inter_apply]
      exact ⟨U.downward_closed hU g, hS⟩
    exact hle (g ≫ f) hgf
```

Status

Item	Description		Status
	Def 50	Causal functor	□ `CausalFunctor` (C ⇒ D)
	Def 51	Heyting algebra	□ Mathlib
	Thm 12	Universal property	□ Proved (□ via Kan ext)
	Thm 13	Cocontinuity	□ Proved (□ via Kan ext)
	Thm 14	Prob has colimits	□ `inferInstance`
	Thm 15	Preserves colimits	□ Proved (Lan adjunction)
	Thm 16	Heyting algebra on topos	□ Proved (explicit himp)

```
import Mathlib.CategoryTheory.Functor.KanExtension.Basic
import Mathlib.MeasureTheory.Measure.Decomposition.RadonNikodym
```

CausalDensity — Definitions 58–59, Theorem 18

Formalizes left Kan extensions, differential causal density, and the RN-Kan duality theorem.

References

- Mahadevan, *Categories for AGI*, Chapter 15 ("Causal Density Functions")

```
open CategoryTheory MeasureTheory
open scoped ENNReal

variable {Ω : Type*} [MeasurableSpace Ω]
```

Definition 58 — Left Kan Extension (restated for causal context)

A left Kan extension of a functor F : C → E along K : C → D is a functor Lan_K F : D → E with a universal natural transformation η : F ⇒ (Lan_K F) ∘ K.

```
#check @Functor.LeftExtension
```

Definition 59 — Differential Causal Density

Let P_obs and P_{do(X_i)} be observational and interventional laws on a measurable space (X, Σ). The differential causal density is defined as the Radon-Nikodym derivative: ρ_i(x) = dP_{do(X_i)} / dP_obs (x) whenever P_{do(X_i)} ≪ P_obs (absolute continuity).

```
/-- Definition 59: Differential causal density as a Radon-Nikodym
 ↪  derivative.
    ρ_i = dP_{do(X_i)} / dP_obs, measuring the pointwise causal effect
    of intervening on variable X_i. -/
noncomputable def differentialCausalDensity
    {α : Type*} [m : MeasurableSpace α]
    (P_obs P_do : @Measure α m) : α → ENNReal :=
  @Measure.rnDeriv α m P_do P_obs
```

Theorem 18 — RN-Kan Duality

Let μ, ν be probability measures on (X, Σ) with ν ≪ μ. Then there is a unique ρ ∈ L^1(μ) such that for all measurable A: ν(A) = ∫_A ρ dμ Moreover, this ρ arises as the "value" of a pointwise left Kan extension in the stochastic category, dualizing the Radon-Nikodym theorem with the categorical Kan extension.

```
-- Theorem 18 (RN-Kan Duality): The Radon-Nikodym derivative ρ = dν/dμ
-- can be interpreted as a pointwise left Kan extension value.
-- The measure-theoretic half uses Mathlib's
 ↪  `Measure.withDensity_rnDeriv_eq`.

/-- Theorem 18 (Radon-Nikodym half): If ν is absolutely continuous w.r.t.
 ↪  μ,
    then ν can be recovered from its Radon-Nikodym derivative:
    μ.withDensity (dν/dμ) = ν. -/
theorem radon_nikodym_density {α : Type*} [MeasurableSpace α]
    (μ ν : Measure α) [ν.HaveLebesgueDecomposition μ]
    (hac : ν ≪ μ) :
    μ.withDensity (ν.rnDeriv μ) = ν :=
  Measure.withDensity_rnDeriv_eq ν μ hac

/-- The categorical Kan-extension interpretation of the Radon-Nikodym
 ↪  theorem:
    the density dν/dμ arises as the unique measurable function making the
    "integration diagram" commute, analogous to a pointwise left Kan
 ↪  extension.
    This structure packages the density together with its defining
 ↪  property. -/
structure RNKanDuality {α : Type*} [MeasurableSpace α]
    (μ ν : Measure α) where
  /-- The density function (Radon-Nikodym derivative) -/
  density : α → ENNReal
  /-- The density recovers the measure: μ.withDensity ρ = ν -/
  density_spec : μ.withDensity density = ν

/-- Construct the RN-Kan duality witness from absolute continuity.
    The Radon-Nikodym derivative is the unique density satisfying the
    Kan-extension universal property (commutativity of the integration
 ↪  diagram). -/
noncomputable def RNKanDuality.ofAbsCont {α : Type*} [MeasurableSpace α]
    (μ ν : Measure α) [ν.HaveLebesgueDecomposition μ]
    (hac : ν ≪ μ) : RNKanDuality μ ν where
  density := ν.rnDeriv μ
  density_spec := Measure.withDensity_rnDeriv_eq ν μ hac

/-- The differential causal density is a special case of RN-Kan duality
    applied to observational and interventional measures. -/
noncomputable def causalDensityAsKan {α : Type*} [MeasurableSpace α]
    (P_obs P_do : Measure α) [P_do.HaveLebesgueDecomposition P_obs]
    (hac : P_do ≪ P_obs) : RNKanDuality P_obs P_do :=
  RNKanDuality.ofAbsCont P_obs P_do hac
```

Status

Item	Description	Status
Def 58	Left Kan extension	□ Mathlib
Def 59	Differential causal density	□ `differentialCausalDensity`
Thm 18	RN-Kan duality	□ `radon_nikodym_density` + `RNKanDuality`

```
import Mathlib.Data.Finset.Basic
```

DoCalculus — Definitions 52–56

Formalizes the classical do-calculus foundations: structural causal models, the do-operator, interventional distributions, potential outcomes, and counterfactuals.

References

- Mahadevan, *Categories for AGI*, Chapter 14 ("Judo Calculus")
- Pearl, *Causality* (2009)

Definition 52 — Structural Causal Model (SCM)

> An SCM is a triple ⟨U, V, F⟩ where V = {V_1, …, V_n} are endogenous variables, U = {U_1, …, U_n} are exogenous variables, and F = {f_1, …, f_n} are structural equations V_i = f_i(pa(V_i), U_i).

```
/-- Definition 52: Structural Causal Model.
    We use a simplified non-dependent formulation where all variables
    share a common value space, avoiding dependent-type complications. -/
structure SCM (Val : Type) where
  /-- Number of variables -/
  n : ℕ
  /-- Parent set for each variable -/
  parents : Fin n → Finset (Fin n)
  /-- Structural equation: variable i's value depends on parent values and
 ↪  noise -/
  mechanism : Fin n → (Fin n → Val) → Val → Val
```

Definition 53 — Do-Operator

> The do-operator do(X = x) produces a modified model M_x by replacing the structural equations for variables in X with the constant x.

```
/-- Definition 53: The do-operator modifies an SCM by surgical
 ↪  intervention.
    do(X_i = x) replaces the mechanism for variable i with a constant. -/
def SCM.doIntervention {Val : Type} (M : SCM Val) (target : Fin M.n)
    (val : Val) : SCM Val where
  n := M.n
  parents := fun i => if i = target then ∅ else M.parents i
  mechanism := fun i parentVals noise =>
    if i = target then val
    else M.mechanism i parentVals noise
```

Definition 54 — Interventional Distribution

The effect of the intervention do(X = x) on a variable Y is the distribution P(Y | do(X = x)), computed by marginalizing over exogenous variables in the modified model M_x.

```
/-- Definition 54: Solve an SCM given exogenous variable values.
    Computes the unique solution to the structural equations by processing
    variables in index order (0, 1, …, n−1).  Under the standard convention
    that parents have strictly lower indices this is a valid topological
    traversal and produces the correct solution. -/
def SCM.solve {Val : Type} [Inhabited Val] (M : SCM Val)
    (exogenous : Fin M.n → Val) : Fin M.n → Val :=
  Fin.foldl M.n (fun acc i =>
    Function.update acc i (M.mechanism i acc (exogenous i)))
    (fun _ => default)
```

Definition 55 — Potential Outcome

The potential outcome of Y in response to an action do(X = x) is the value Y would take in the modified model M_x, denoted Y_x(u) for a given realization of exogenous variables u.

```
/-- Definition 55: Potential outcome Y_x(u).
    The value that variable `outcome` would take if we intervene to set
    `target` to `val`, given exogenous values `exogenous`.
    Formally: Y_x(u) = solve(M_x, u)(outcome). -/
def SCM.potentialOutcome {Val : Type} [Inhabited Val] (M : SCM Val) (target
 ↪  : Fin M.n)
    (val : Val) (exogenous : Fin M.n → Val) (outcome : Fin M.n) : Val :=
  (M.doIntervention target val).solve exogenous outcome
```

Definition 56 — Counterfactual

The counterfactual sentence “The value that Y would have obtained had X been x, given that what we actually observed was e” is defined via the three-step abduction-action-prediction procedure.

```
/-- Definition 56: Counterfactual query.
    Captures the three-step abduction-action-prediction procedure:
    1. **Abduction**: Given evidence, infer posterior over exogenous
 ↪  variables U.
    2. **Action**: Apply the intervention do(X = x) to get modified model
 ↪  M_x.
    3. **Prediction**: Compute the outcome Y in M_x using the inferred U.
 ↪  -/
structure Counterfactual (Val : Type) where
  /-- The base SCM -/
  model : SCM Val
```

```
  /-- Evidence: observed variable-value pairs (variable index, observed
 ↪  value) -/
  evidence : List (Fin model.n × Val)
  /-- Intervention target variable -/
  target : Fin model.n
  /-- Intervention value -/
  interventionVal : Val
  /-- Outcome variable of interest -/
  outcome : Fin model.n

/-- Evaluate a counterfactual given a specific exogenous assignment.
    In the deterministic case (known U), the three-step process reduces to:
    solve(M_{do(X=x)}, u)(Y). -/
def Counterfactual.eval {Val : Type} [Inhabited Val] (cf : Counterfactual
 ↪  Val)
    (exogenous : Fin cf.model.n → Val) : Val :=
  cf.model.potentialOutcome cf.target cf.interventionVal exogenous
 ↪  cf.outcome
```

Status

Def	Description	Status
52	SCM	✅ `SCM` structure
53	Do-operator	✅ `SCM.doIntervention`
54	Interventional dist	✅ `SCM.solve`
55	Potential outcome	✅ `SCM.potentialOutcome`
56	Counterfactual	✅ `Counterfactual`

```
import Mathlib.CategoryTheory.Sites.Grothendieck
import Mathlib.Order.Heyting.Basic
```

JudoCalculus — Definition 57, Theorem 17

Formalizes the Lawvere-Tierney causal topology and the j-do calculus rules.

References

- Mahadevan, *Categories for AGI*, Chapter 14 ("Judo Calculus")

```
open CategoryTheory
```

Definition 57 — Lawvere-Tierney Causal Topology

> Let E be an elementary topos with subobject classifier Ω. A Lawvere-Tierney topology is a morphism j : Ω → Ω satisfying: (1) j ∘ true = true (2) j ∘ j = j (idempotent) (3) j ∘ ∧ = ∧ ∘ (j × j) (preserves meets)

```
/-- Definition 57: Lawvere-Tierney topology as an idempotent,
    meet-preserving closure operator on the subobject classifier.
    In a Heyting algebra H, this is j : H → H with j(⊤) = ⊤,
    j ∘ j = j, j(a ∧ b) = j(a) ∧ j(b). -/
structure LawvereTierneyTopology (H : Type*) [HeytingAlgebra H] where
  j : H → H
  preserves_top : j ⊤ = ⊤
  idempotent : ∀ a, j (j a) = j a
  preserves_meet : ∀ a b, j (a □ b) = j a □ j b
```

Theorem 17 — Grothendieck ↔ Lawvere-Tierney

> If C is a small category, the Grothendieck topologies J on C correspond bijectively to Lawvere-Tierney topologies j on the presheaf topos Sets$^{\{C}$op}.

```
/-- Theorem 17: Every Grothendieck topology J on C induces a closure
 ↪  operator
    on sieves satisfying the Lawvere-Tierney axioms (idempotent, preserves
 ↪  top,
    preserves meets). The converse also holds: every LT topology on the
    presheaf topos arises from a unique Grothendieck topology.
    We state one direction: Grothendieck → Lawvere-Tierney. -/
theorem grothendieck_eq_lawvere_tierney
    {C : Type*} [SmallCategory C] (J : GrothendieckTopology C) (X : C) :
    ∃ (cl : Sieve X → Sieve X),
      (cl ⊤ = ⊤) ∧
      (∀ S, cl (cl S) = cl S) ∧
      (∀ S T, cl (S □ T) = cl S □ cl T) := by
  -- cl(S)(f) := S.pullback f ∈ J Y
  refine ⟨fun S => {
```

```
    arrows := fun {Y} (f : Y ⟶ X) => S.pullback f ∈ J Y
    downward_closed := by
      intro Y Z f hf g
      rw [Sieve.pullback_comp]
      exact J.pullback_stable g hf }, ?_, ?_, ?_⟩
  -- (1) cl(⊤) = ⊤
  · apply Sieve.ext; intro Y f; constructor
    · intro _; trivial
    · intro _
      show (⊤ : Sieve X).pullback f ∈ J Y
      rw [Sieve.pullback_top]; exact J.top_mem Y
  -- (2) Idempotent: cl(cl S) = cl S
  · intro S; apply Sieve.ext; intro Y f; constructor
    · -- ⇒: use J.transitive
      intro hcl
      apply J.transitive hcl (S.pullback f)
      intro Z g hg
      rw [← Sieve.pullback_comp]
      exact hg
    · -- ⇐: use J.superset_covering
      intro hcl
      apply J.superset_covering _ hcl
      intro Z g _
      show S.pullback (g ≫ f) ∈ J Z
      rw [Sieve.pullback_comp]
      exact J.pullback_stable g hcl
  -- (3) Preserves meets: cl(S ⊓ T) = cl(S) ⊓ cl(T)
  · intro S T; apply Sieve.ext; intro Y f
    simp only [Sieve.inter_apply]; constructor
    · -- ⇒: covering of intersection implies each part covers
      intro h; rw [Sieve.pullback_inter] at h
      exact ⟨J.superset_covering inf_le_left h,
             J.superset_covering inf_le_right h⟩
    · -- ⇐: both covering implies intersection covers
      intro ⟨hS, hT⟩; rw [Sieve.pullback_inter]
      apply J.transitive hS
      intro Z g hg
      rw [Sieve.pullback_inter]
      have hSgf : S (g ≫ f) := hg
      rw [← Sieve.pullback_comp, ← Sieve.pullback_comp,
          Sieve.pullback_eq_top_of_mem S hSgf, top_inf_eq,
↪  Sieve.pullback_comp]
      exact J.pullback_stable g hT
```

j-do Rules

[j-Rule 1]: Insert/delete observations — if (Y ⫫ Z | X)*{G_X̄} then P(y | do(x), z) = P(y | do(x)) [j-Rule 2]: Action/observation exchange — if (Y ⫫ Z |*

X){G_{X̄,Z̲}} then P(y | do(x), do(z)) = P(y | do(x), z) [j-Rule 3]: Insert/delete actions — if (Y ⊥⊥ Z | X)*{G*{X̄,Z̄(S)}} then P(y | do(x), do(z)) = P(y | do(x))

```
/-- j-Rule 1 (insert/delete observations): If a condition c is j-dense
    (j(c) = ⊤, modeling conditional independence), conjoining c does not
    change the j-closure. Models: P(y | do(x), z) = P(y | do(x)). -/
theorem j_rule_insert_delete {H : Type*} [HeytingAlgebra H]
    (τ : LawvereTierneyTopology H) (c a : H)
    (hc : τ.j c = ⊤) :
    τ.j (c ⊓ a) = τ.j a := by
  rw [τ.preserves_meet, hc, top_inf_eq]

/-- j-Rule 2 (action/observation exchange): If an element is j-closed
    (an "action") and a condition is j-dense (an "observation"),
    their conjunction j-closes to the action.
    Models: P(y | do(x), do(z)) = P(y | do(x), z). -/
theorem j_rule_exchange {H : Type*} [HeytingAlgebra H]
    (τ : LawvereTierneyTopology H) (action condition : H)
    (h_closed : τ.j action = action) (h_dense : τ.j condition = ⊤) :
    τ.j (condition ⊓ action) = action := by
  rw [τ.preserves_meet, h_dense, top_inf_eq, h_closed]

/-- j-Rule 3 (insert/delete actions): The conjunction of j-dense elements
    is j-dense. Models: P(y | do(x), do(z)) = P(y | do(x))
    when both interventions are conditionally independent. -/
theorem j_rule_insert_delete_actions {H : Type*} [HeytingAlgebra H]
    (τ : LawvereTierneyTopology H) (a b : H)
    (ha : τ.j a = ⊤) (hb : τ.j b = ⊤) :
    τ.j (a ⊓ b) = ⊤ := by
  rw [τ.preserves_meet, ha, hb, top_inf_eq]
```

Conservativity Proposition

With J_{id} (the identity topology), j-do calculus reduces exactly to Pearl's classical do-calculus.

```
/-- Conservativity: the identity Lawvere-Tierney topology yields
    classical do-calculus. -/
def identityLT (H : Type*) [HeytingAlgebra H] : LawvereTierneyTopology H
 ↪ where
  j := id
  preserves_top := rfl
  idempotent := fun _ => rfl
  preserves_meet := fun _ _ => rfl
```

Status

Item	Description	Status
Def 57	Lawvere-Tierney topology	✓ `LawvereTierneyTopology`
Thm 17	Groth ↔ LT correspondence	✓ Proved (closure operator)
j-Rule1	Insert/delete observations	✓ `j_rule_insert_delete`
j-Rule2	Action/observation exchange	✓ `j_rule_exchange`
j-Rule3	Insert/delete actions	✓ `j_rule_insert_delete_actions`
Conserv	Identity = classical do-calc	✓ `identityLT`

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Functor.Basic
```

Coalgebras — Definitions 60–62

Formalizes labeled transition systems, coalgebras, and the probability distribution functor for modeling consciousness.

References

- Mahadevan, *Categories for AGI*, Chapter 16 ("Consciousness")

```
open CategoryTheory
```

Definition 60 — Labeled Transition System (LTS)

> A labeled transition system (S, →_S, A) is defined by a set S of states, a transition relation →_S ⊆ S × A × S, and a set A of action labels.

```
/-- Definition 60: Labeled Transition System. -/
structure LTS where
  /-- State space -/
  S : Type
  /-- Action labels -/
  A : Type
  /-- Transition relation: s --a--> s' -/
  trans : S → A → S → Prop
```

Definition 61 — Coalgebra for a Functor

> Let F : C → C be an endofunctor. An F-coalgebra is a pair (X, α) where X is an object and α : X → F(X) is a morphism.

```
/-- Definition 61: F-coalgebra for an endofunctor F. -/
structure Coalgebra {C : Type*} [Category C] (F : C ⥤ C) where
  carrier : C
  struct : carrier ⟶ F.obj carrier

/-- Morphism of F-coalgebras. -/
structure CoalgebraMor {C : Type*} [Category C] {F : C ⥤ C}
    (A B : Coalgebra F) where
  hom : A.carrier ⟶ B.carrier
  comm : hom ≫ B.struct = A.struct ≫ F.map hom

/-- Extensionality for coalgebra morphisms: determined by their carrier
 ↪ morphism. -/
@[ext]
theorem CoalgebraMor.ext {C : Type*} [Category C] {F : C ⥤ C} {A B :
 ↪ Coalgebra F}
```

```
    {f g : CoalgebraMor A B} (h : f.hom = g.hom) : f = g := by
  rcases f with ⟨_, _⟩; rcases g with ⟨_, _⟩; subst h; rfl

/-- The category of F-coalgebras for an endofunctor F : C ⥤ C.
    Objects are F-coalgebras and morphisms commute with the structure maps.
↪  -/
instance coalgebraCategory {C : Type*} [Category C] (F : C ⥤ C) :
    Category (Coalgebra F) where
  Hom := CoalgebraMor
  id A := ⟨𝟙 A.carrier, by simp⟩
  comp f g := ⟨f.hom ≫ g.hom, by
    rw [Category.assoc, g.comm, ← Category.assoc, f.comm, Category.assoc, ←
↪  F.map_comp]⟩
  id_comp f := by ext; simp
  comp_id f := by ext; simp
  assoc f g h := by ext; simp [Category.assoc]
```

Definition 62 — Probability Distribution Functor

The probability distribution functor D is defined as D : Meas → Meas mapping a measurable space X to the space D(X) of probability measures on X.

```
/-- Definition 62: The probability distribution functor.
    Maps a measurable space to its space of probability measures,
    and a measurable function to its pushforward on measures. -/
structure ProbDistFunctor where
  /-- On objects: X ↦ Prob(X) -/
  onObj : Type → Type
  /-- On morphisms: measurable f ↦ pushforward f_* -/
  onMor : ∀ {X Y : Type}, (X → Y) → onObj X → onObj Y
  /-- Functoriality: identity maps are preserved. -/
  onMor_id : ∀ {X : Type}, onMor (id : X → X) = id
  /-- Functoriality: composition is preserved. -/
  onMor_comp : ∀ {X Y Z : Type} (f : X → Y) (g : Y → Z),
    onMor (g ∘ f) = onMor g ∘ onMor f
```

Stochastic Coalgebras

A stochastic coalgebra is a D-coalgebra, i.e., a pair (X, α : X → D(X)). This models a probabilistic transition system where from each state there is a probability distribution over next states.

```
/-- A stochastic coalgebra (X, α : X → D(X)) models probabilistic dynamics.
    Used in the consciousness framework to model state transitions
    of conscious/unconscious processes. -/
structure StochasticCoalgebra (D : ProbDistFunctor) where
  /-- State space -/
  X : Type
```

```
  /-- Transition kernel: each state maps to a distribution over next states
 ↪  -/
  α : X → D.onObj X

/-- A morphism of stochastic coalgebras: a measurable map between state
 ↪  spaces
    that commutes with the transition kernels. -/
structure StochasticCoalgebraMor (D : ProbDistFunctor) (A B :
 ↪  StochasticCoalgebra D) where
  /-- Map on state spaces -/
  hom : A.X → B.X
  /-- Commutativity: D(hom) ∘ α_A = α_B ∘ hom -/
  comm : D.onMor hom ∘ A.α = B.α ∘ hom
```

Status

Def	Description		Status
	60	LTS	✅ `LTS`
	61	Coalgebra	✅ `Coalgebra`
	—	Coalgebra category	✅ `coalgebraCategory`
	62	Prob distribution func	✅ `ProbDistFunctor`
	—	Stochastic coalgebra	✅ `StochasticCoalgebra`

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Monoidal.Category
import Mathlib.CategoryTheory.Functor.Basic
import Mathlib.Logic.Equiv.Defs
```

LearnCategory — Definitions 31–32 from *Categories for AGI*

Formalizes the category **Learn** of supervised learners and the category **Param** of parameterized functions, including backpropagation as a functor.

Morphisms are equivalence classes of learners (respectively, parameterized functions), quotiented by parameter-space reparametrization so that the category axioms hold exactly.

References

- Mahadevan, *Categories for AGI*, Chapter 5 ("Categorical Deep Learning")
- Fong, Spivak, Tuyéras, "Backprop as Functor" (2019)

```
open CategoryTheory
```

Definition 31 — Category Learn

> The symmetric monoidal category Learn is defined as: - Objects: sets (input/output spaces) - Morphisms (A, B): triples (P, I, U, r) where P is a parameter space, I : A × P → B is the implementation, U : A × B × P → P is the update (learning) rule, and r : B × B → ℝ is the request function.
>
> Morphisms are taken up to reparametrization of the parameter space.

```
/-- Definition 31: A learner morphism in the category Learn.
    Captures parameterized implementation + update rule. -/
structure Learner (A B : Type) where
  /-- Parameter space -/
  P : Type
  /-- Implementation: given input and parameters, produce output -/
  impl : A × P → B
  /-- Update rule: given input, desired output, and parameters, update
 ↪  parameters -/
  update : A × B × P → P
  /-- Request function: compares actual vs desired output -/
  request : B × B → Float

/-- Composition of learners (Definition 31).
    Given learners f : A → B and g : B → C, their composite
    uses the chain rule for parameter updates. -/
def Learner.comp {A B C : Type} (f : Learner A B) (g : Learner B C) :
    Learner A C where
  P := f.P × g.P
```

```
  impl := fun ⟨a, p_f, p_g⟩ => g.impl ⟨f.impl ⟨a, p_f⟩, p_g⟩
  update := fun ⟨a, c, p_f, p_g⟩ =>
    let b := f.impl ⟨a, p_f⟩
    (f.update ⟨a, b, p_f⟩, g.update ⟨b, c, p_g⟩)
  request := g.request

/-- Identity learner: passes input through unchanged. -/
def Learner.id (A : Type) : Learner A A where
  P := PUnit
  impl := fun ⟨a, _⟩ => a
  update := fun ⟨_, _, p⟩ => p
  request := fun ⟨_, _⟩ => 0.0
```

Equivalence relation on learners

```
/-- Equivalence of learners up to parameter-space reparametrization.
    Two learners are equivalent when their implementations agree
    modulo a bijection on parameter spaces. -/
def Learner.Equiv {A B : Type} (f g : Learner A B) : Prop :=
  ∃ (e : f.P ≃ g.P), ∀ a p, f.impl (a, p) = g.impl (a, e p)

/-- The equivalence relation on learners forms a setoid. -/
instance learnerSetoid (A B : Type) : Setoid (Learner A B) where
  r := Learner.Equiv
  iseqv := {
    refl := fun f => ⟨.refl f.P, fun _ _ => rfl⟩
    symm := fun {f g} ⟨e, h⟩ => ⟨e.symm, fun a p => by
      have := (h a (e.symm p)).symm
      rwa [Equiv.apply_symm_apply] at this⟩
    trans := fun {_ _ _} ⟨e₁, h₁⟩ ⟨e₂, h₂⟩ => ⟨e₁.trans e₂, fun a p => by
      simp only [Equiv.trans_apply]; exact (h₁ a p).trans (h₂ a (e₁ p))⟩
  }
```

Canonical parameter-space equivalences

```
private def prodCongrEquiv {α₁ α₂ β₁ β₂ : Type}
    (e₁ : α₁ ≃ α₂) (e₂ : β₁ ≃ β₂) : α₁ × β₁ ≃ α₂ × β₂ where
  toFun := fun ⟨a, b⟩ => ⟨e₁ a, e₂ b⟩
  invFun := fun ⟨a, b⟩ => ⟨e₁.symm a, e₂.symm b⟩
  left_inv := fun ⟨a, b⟩ => by simp
  right_inv := fun ⟨a, b⟩ => by simp

private def punitProdEquiv (α : Type) : PUnit × α ≃ α where
  toFun := fun ⟨_, a⟩ => a
  invFun := fun a => ⟨PUnit.unit, a⟩
  left_inv := fun ⟨u, _⟩ => by cases u; rfl
  right_inv := fun _ => rfl
```

```
private def prodPUnitEquiv (α : Type) : α × PUnit ≃ α where
  toFun := fun ⟨a, _⟩ => a
  invFun := fun a => ⟨a, PUnit.unit⟩
  left_inv := fun ⟨_, u⟩ => by cases u; rfl
  right_inv := fun _ => rfl

private def prodAssocEquiv (α β γ : Type) : (α × β) × γ ≃ α × (β × γ) where
  toFun := fun ⟨⟨a, b⟩, c⟩ => ⟨a, b, c⟩
  invFun := fun ⟨a, b, c⟩ => ⟨⟨a, b⟩, c⟩
  left_inv := fun ⟨⟨_, _⟩, _⟩ => rfl
  right_inv := fun ⟨_, _, _⟩ => rfl

/-- Composition of learners respects the equivalence relation. -/
private theorem Learner.comp_equiv {A B C : Type}
    ⦃f₁ f₂ : Learner A B⦄ (hf : f₁ ≈ f₂)
    ⦃g₁ g₂ : Learner B C⦄ (hg : g₁ ≈ g₂) :
    Learner.comp f₁ g₁ ≈ Learner.comp f₂ g₂ := by
  obtain ⟨ef, hf⟩ := hf
  obtain ⟨eg, hg⟩ := hg
  refine ⟨prodCongrEquiv ef eg, fun a ⟨pf, pg⟩ => ?_⟩
  show g₁.impl (f₁.impl (a, pf), pg) = g₂.impl (f₂.impl (a, ef pf), eg pg)
  rw [hf a pf, hg _ pg]

/-- Wrapper type for objects in the Learn category, avoiding conflict with
    the existing category instance on Type. -/
structure LearnObj where
  carrier : Type

/-- Category instance for Learn. Morphisms are equivalence classes of
↪  learners
    under parameter-space reparametrization. -/
instance : Category LearnObj where
  Hom A B := Quotient (learnerSetoid A.carrier B.carrier)
  id A := ⟦Learner.id A.carrier⟧
  comp f g := Quotient.map₂ Learner.comp
    (fun _ _ hf _ _ hg => Learner.comp_equiv hf hg) f g
  id_comp f := Quotient.inductionOn f fun f => by
    simp only [Quotient.map₂_mk]
    exact Quotient.sound ⟨punitProdEquiv f.P, fun a ⟨u, p⟩ => by cases u;
↪  rfl⟩
  comp_id f := Quotient.inductionOn f fun f => by
    simp only [Quotient.map₂_mk]
    exact Quotient.sound ⟨prodPUnitEquiv f.P, fun a ⟨p, u⟩ => by cases u;
↪  rfl⟩
  assoc f g h := Quotient.inductionOn f fun f =>
    Quotient.inductionOn g fun g =>
      Quotient.inductionOn h fun h => by
    simp only [Quotient.map₂_mk]
```

```
    exact Quotient.sound ⟨prodAssocEquiv f.P g.P h.P,
      fun a ⟨⟨pf, pg⟩, ph⟩ => rfl⟩
```

Definition 32 — Category Param

The category Param defines a strict symmetric monoidal category whose objects are Euclidean spaces, and whose morphisms are smooth parameterized functions with their derivatives.

```
/-- Definition 32: A morphism in the Param category.
    A differentiable parameterized function with its derivative. -/
structure ParamMor (A B : Type) where
  /-- Parameter space -/
  P : Type
  /-- Forward map -/
  forward : P → A → B
  /-- Derivative / Jacobian (for backpropagation) -/
  backward : P → A → B → A × P

/-- Identity parameterized morphism. -/
def ParamMor.id (A : Type) : ParamMor A A where
  P := PUnit
  forward := fun _ a => a
  backward := fun _ _ b => ⟨b, PUnit.unit⟩

/-- Composition of parameterized morphisms (chain rule). -/
def ParamMor.comp {A B C : Type} (f : ParamMor A B) (g : ParamMor B C) :
    ParamMor A C where
  P := f.P × g.P
  forward := fun ⟨pf, pg⟩ a => g.forward pg (f.forward pf a)
  backward := fun ⟨pf, pg⟩ a c =>
    let b := f.forward pf a
    let (b_grad, pg') := g.backward pg b c
    let (a_grad, pf') := f.backward pf a b_grad
    (a_grad, (pf', pg'))
```

Equivalence relation on parameterized morphisms

```
/-- Equivalence of parameterized morphisms up to parameter-space
    reparametrization. Two morphisms are equivalent when their forward
    maps agree modulo a bijection on parameter spaces. -/
def ParamMor.Equiv {A B : Type} (f g : ParamMor A B) : Prop :=
  ∃ (e : f.P ≃ g.P), ∀ p a, f.forward p a = g.forward (e p) a

/-- The equivalence relation on parameterized morphisms forms a setoid. -/
instance paramMorSetoid (A B : Type) : Setoid (ParamMor A B) where
  r := ParamMor.Equiv
  iseqv := {
    refl := fun f => ⟨.refl f.P, fun _ _ => rfl⟩
```

```
    symm := fun {f g} ⟨e, h⟩ => ⟨e.symm, fun p a => by
      have := (h (e.symm p) a).symm
      rwa [Equiv.apply_symm_apply] at this⟩
    trans := fun {_ _ _} ⟨e₁, h₁⟩ ⟨e₂, h₂⟩ => ⟨e₁.trans e₂, fun p a => by
      simp only [Equiv.trans_apply]; exact (h₁ p a).trans (h₂ (e₁ p) a)⟩
  }

/-- Composition of parameterized morphisms respects the equivalence. -/
private theorem ParamMor.comp_equiv {A B C : Type}
    ⦃f₁ f₂ : ParamMor A B⦄ (hf : f₁ ≈ f₂)
    ⦃g₁ g₂ : ParamMor B C⦄ (hg : g₁ ≈ g₂) :
    ParamMor.comp f₁ g₁ ≈ ParamMor.comp f₂ g₂ := by
  obtain ⟨ef, hf⟩ := hf
  obtain ⟨eg, hg⟩ := hg
  refine ⟨prodCongrEquiv ef eg, fun ⟨pf, pg⟩ a => ?_⟩
  show g₁.forward pg (f₁.forward pf a) = g₂.forward (eg pg) (f₂.forward (ef
↪  pf) a)
  rw [hf pf a, hg pg _]

/-- Wrapper type for objects in the Param category. -/
structure ParamObj where
  carrier : Type

/-- Category instance for Param. Morphisms are equivalence classes of
    parameterized functions under parameter-space reparametrization. -/
instance : Category ParamObj where
  Hom A B := Quotient (paramMorSetoid A.carrier B.carrier)
  id A := ⟦ParamMor.id A.carrier⟧
  comp f g := Quotient.map₂ ParamMor.comp
    (fun _ _ hf _ _ hg => ParamMor.comp_equiv hf hg) f g
  id_comp f := Quotient.inductionOn f fun f => by
    simp only [Quotient.map₂_mk]
    exact Quotient.sound ⟨punitProdEquiv f.P, fun ⟨u, p⟩ a => by cases u;
↪  rfl⟩
  comp_id f := Quotient.inductionOn f fun f => by
    simp only [Quotient.map₂_mk]
    exact Quotient.sound ⟨prodPUnitEquiv f.P, fun ⟨p, u⟩ a => by cases u;
↪  rfl⟩
  assoc f g h := Quotient.inductionOn f fun f =>
    Quotient.inductionOn g fun g =>
      Quotient.inductionOn h fun h => by
    simp only [Quotient.map₂_mk]
    exact Quotient.sound ⟨prodAssocEquiv f.P g.P h.P,
      fun ⟨⟨pf, pg⟩, ph⟩ a => rfl⟩
```

Backpropagation as a Functor

The backpropagation algorithm defines a functor from Param to Learn: it sends each parameterized differentiable function to a learner whose update rule is gradient descent.

```
/-- Map a parameterized morphism to a learner via backpropagation.
    The forward pass becomes the implementation, and the backward pass
    (derivative) provides the parameter update rule (gradient descent). -/
def backpropMap {A B : Type} (f : ParamMor A B) : Learner A B where
  P := f.P
  impl := fun ⟨a, p⟩ => f.forward p a
  update := fun ⟨a, b, p⟩ => (f.backward p a b).2
  request := fun ⟨_, _⟩ => 0.0

/-- backpropMap respects the equivalence on parameterized morphisms. -/
private theorem backpropMap_respects {A B : Type}
    ⦃f₁ f₂ : ParamMor A B⦄ (h : f₁ ≈ f₂) :
    @Setoid.r _ (learnerSetoid A B) (backpropMap f₁) (backpropMap f₂) := by
  obtain ⟨e, he⟩ := h
  exact ⟨e, fun a p => he p a⟩

/-- Backpropagation as a functor Param → Learn.
    The key insight is that the chain rule for derivatives corresponds
    to functorial composition of learners: backprop(g ∘ f) ≅ backprop(g) ∘
 ↪  backprop(f). -/
def backpropFunctor : ParamObj ⥤ LearnObj where
  obj A := ⟨A.carrier⟩
  map f := Quotient.map backpropMap (fun _ _ h => backpropMap_respects h) f
  map_id X := by
    exact Quotient.sound ⟨.refl _, fun a p => by cases p; rfl⟩
  map_comp f g := Quotient.inductionOn f fun f =>
    Quotient.inductionOn g fun g => by
    simp only [Quotient.map_mk]
    exact Quotient.sound ⟨.refl _, fun a ⟨pf, pg⟩ => rfl⟩
```

Status

Def	Description	Status
31	Category Learn	✓ `LearnObj` + `Category` (quotient)
32	Category Param	✓ `ParamObj` + `Category` (quotient)
BP	Backprop functor	✓ `backpropFunctor` (fully proved)

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Monoidal.Category
import Mathlib.CategoryTheory.Enriched.Basic
```

TransformerCategory — Definitions 26–30 from *Categories for AGI*

Novel categorical definitions for Transformer models: the Transformer block as a morphism, the category C_T of Transformer models, LLM syntax and semantic categories, and the k-NN LLM category.

References

- Mahadevan, *Categories for AGI*, Chapter 5 ("Categorical Deep Learning")

```
open CategoryTheory
```

Definition 26 — Transformer Block

> A Transformer block is a sequence-to-sequence function mapping ℝ^{d × n} → ℝ^{d × n}. There are generally two types: encoder blocks and decoder blocks (which include causal masking).

```
/-- Definition 26: A Transformer block as a sequence-to-sequence map.
    We model it abstractly as an endomorphism on a sequence space. -/
structure TransformerBlock (d n : ℕ) where
  /-- The forward map of the block -/
  forward : (Fin d → Fin n → Float) → (Fin d → Fin n → Float)

/-- Identity transformer block (passes input through unchanged). -/
def TransformerBlock.id (d n : ℕ) : TransformerBlock d n where
  forward := _root_.id

/-- Compose two transformer blocks (feed-forward composition). -/
def TransformerBlock.comp {d n : ℕ} (f g : TransformerBlock d n) :
 ↪  TransformerBlock d n where
  forward := g.forward ∘ f.forward
```

Definition 27 — Category C_T of Transformer Models

> The category C_T of Transformer models has objects that are sequence spaces ℝ^{d × n} and morphisms that are Transformer blocks (compositions of self-attention and feed-forward layers).

```
/-- Definition 27: Objects of the Transformer category are
    (dimension, sequence length) pairs. -/
structure TransObj where
  dim : ℕ
```

```
  seqLen : ℕ

/-- Definition 27: The category C_T of Transformer models.
    Morphisms are Transformer blocks. -/
structure TransMor (X Y : TransObj) where
  /-- Underlying function between sequence spaces -/
  toFun : (Fin X.dim → Fin X.seqLen → Float) →
          (Fin Y.dim → Fin Y.seqLen → Float)

/-- Identity morphism in the Transformer category. -/
def TransMor.id (X : TransObj) : TransMor X X where
  toFun := _root_.id

/-- Composition of morphisms in the Transformer category. -/
def TransMor.comp {X Y Z : TransObj} (f : TransMor X Y) (g : TransMor Y Z)
 ↪  : TransMor X Z where
  toFun := g.toFun ∘ f.toFun

/-- Extensionality for Transformer morphisms: two morphisms are equal
    iff their underlying functions are equal. -/
@[ext]
theorem TransMor.ext {X Y : TransObj} {f g : TransMor X Y}
    (h : f.toFun = g.toFun) : f = g := by
  cases f; cases g; subst h; rfl

/-- The category C_T of Transformer models, with sequence spaces as objects
    and sequence-to-sequence maps as morphisms. -/
instance : Category TransObj where
  Hom := TransMor
  id := TransMor.id
  comp := TransMor.comp
  id_comp f := by ext; rfl
  comp_id f := by ext; rfl
  assoc f g h := by ext; rfl
```

Definition 28 — LLM Syntax Category

> The LLM syntax category L is defined as a category enriched over a monoidal category of probability distributions, where morphisms L(y | x) represent next-token conditional distributions.

```
/-- Definition 28: Objects of the LLM syntax category.
    Each object is a language model with a vocabulary size and context
 ↪  length. -/
structure LLMSynObj where
  vocabSize : ℕ
  ctxLen : ℕ

/-- Definition 28: Morphisms in the LLM syntax category.
```

```
    A stochastic map (conditional distribution) between token sequence
 ↪  spaces,
    modeled as a function on sequence representations. -/
structure LLMSynMor (X Y : LLMSynObj) where
  /-- The underlying deterministic map between sequence representations -/
  toFun : (Fin X.vocabSize → Fin X.ctxLen → Float) →
          (Fin Y.vocabSize → Fin Y.ctxLen → Float)

/-- Extensionality for LLM syntax morphisms. -/
@[ext]
theorem LLMSynMor.ext {X Y : LLMSynObj} {f g : LLMSynMor X Y}
    (h : f.toFun = g.toFun) : f = g := by
  cases f; cases g; subst h; rfl

/-- The LLM syntax category: objects are (vocab, context) pairs,
    morphisms are maps between sequence spaces with identity and
 ↪  composition. -/
instance : Category LLMSynObj where
  Hom := LLMSynMor
  id X := ⟨id⟩
  comp f g := ⟨g.toFun ∘ f.toFun⟩
  id_comp f := by ext; rfl
  comp_id f := by ext; rfl
  assoc f g h := by ext; rfl

/-- Original scaffold structure preserved for backward compatibility. -/
structure LLMSyntaxCat where
  Token : Type
  /-- Conditional distribution P(next | context) -/
  nextTokenDist : (List Token) → Token → Float
```

Definition 29 — LLM Semantic Category

For the LLM category L, the semantic category L_sem is defined with objects that are meaning representations and morphisms that capture semantic entailment relationships.

```
/-- Definition 29: LLM semantic category.
    Objects are meaning representations, with an entailment preorder.
    Morphisms are entailment proofs. -/
structure LLMSemanticCat where
  Meaning : Type
  entails : Meaning → Meaning → Prop
  /-- Entailment is reflexive -/
  entails_refl : ∀ x, entails x x
  /-- Entailment is transitive -/
  entails_trans : ∀ x y z, entails x y → entails y z → entails x z
```

```
/-- The entailment relation on an LLM semantic category forms a preorder.
 ↪  -/
@[reducible]
def LLMSemanticCat.toPreorder (S : LLMSemanticCat) : Preorder S.Meaning
 ↪  where
  le := S.entails
  le_refl := S.entails_refl
  le_trans := S.entails_trans
```

Definition 30 — k-NN LLM Syntax Category

> The k-NN LLM syntax category L_{kNN} is defined as a category whose morphisms L_{kNN}(y | x) combine parametric and non-parametric (k-nearest-neighbor) retrieval-based next-token distributions.

```
/-- Definition 30: k-NN augmented LLM category.
    Combines parametric and retrieval-based distributions. -/
structure KNNLLMCat where
  Token : Type
  /-- Parametric next-token distribution -/
  parametricDist : (List Token) → Token → Float
  /-- k-NN retrieval distribution -/
  retrievalDist : (List Token) → Token → Float
  /-- Interpolation weight λ ∈ [0, 1] -/
  lambda : Float

/-- The combined k-NN LLM distribution: λ · retrieval + (1 − λ) ·
 ↪  parametric.
    This is the interpolation formula from Khandelwal et al. (2020). -/
def KNNLLMCat.combinedDist (m : KNNLLMCat) (ctx : List m.Token) (tok :
 ↪  m.Token) : Float :=
  m.lambda * m.retrievalDist ctx tok + (1 - m.lambda) * m.parametricDist
 ↪  ctx tok

/-- When λ = 0, the combined distribution equals the parametric
 ↪  distribution. -/
theorem KNNLLMCat.combinedDist_lambda_zero (m : KNNLLMCat) (h : m.lambda =
 ↪  0)
    (ctx : List m.Token) (tok : m.Token) :
    m.combinedDist ctx tok = 0 * m.retrievalDist ctx tok +
      (1 - 0) * m.parametricDist ctx tok := by
  simp [KNNLLMCat.combinedDist, h]

/-- When λ = 1, the combined distribution equals the retrieval
 ↪  distribution. -/
theorem KNNLLMCat.combinedDist_lambda_one (m : KNNLLMCat) (h : m.lambda =
 ↪  1)
    (ctx : List m.Token) (tok : m.Token) :
    m.combinedDist ctx tok = 1 * m.retrievalDist ctx tok +
```

```
    (1 - 1) * m.parametricDist ctx tok := by
  simp [KNNLLMCat.combinedDist, h]
```

Status

Def	Description	Status
26	Transformer block	⬜ Struct + id/comp
27	Category C_T	⬜ Category instance
28	LLM syntax cat	⬜ `LLMSynObj` + `Category`
29	LLM semantic cat	⬜ `LLMSemanticCat` + `Preorder`
30	k-NN LLM cat	⬜ `KNNLLMCat` + `combinedDist` + λ-boundary thms

```
import Mathlib.Analysis.Normed.Group.Basic
import Mathlib.MeasureTheory.Measure.MeasureSpace
import Mathlib.MeasureTheory.Integral.Bochner.Basic
```

DynamicCompositionality — Definition 41

Formalizes dynamic compositionality, commutator energy, and the Čech-style obstruction proxy.

References

- Mahadevan, *Categories for AGI*, Chapter 8 ("Dynamic Compositionality")

Definition 41 — Dynamic Compositionality

> Dynamic compositionality is the property that the order in which sub-operators (e.g., attention, feed-forward) are applied matters for the learned representation, and this order sensitivity can be measured by the commutator energy.

Commutator Energy

> Given two operators F, G acting on a representation space, the commutator energy is E_comm = $\|F \circ G - G \circ F\|^2$. This measures how much the result depends on application order.

```
/-- Commutator energy of two endomorphisms on a normed space.
    E_comm(F, G, x) = ‖F(G(x)) - G(F(x))‖² -/
noncomputable def commutatorEnergy {V : Type*} [NormedAddCommGroup V]
    (F G : V → V) (x : V) : ℝ :=
  ‖F (G x) - G (F x)‖ ^ 2

/-- Commutator energy is non-negative (it is a squared norm). -/
theorem commutatorEnergy_nonneg {V : Type*} [NormedAddCommGroup V]
    (F G : V → V) (x : V) : 0 ≤ commutatorEnergy F G x :=
  sq_nonneg _

/-- Commutator energy is symmetric in F and G: swapping the operators
    does not change the energy, since ‖a - b‖ = ‖b - a‖. -/
theorem commutatorEnergy_symm {V : Type*} [NormedAddCommGroup V]
    (F G : V → V) (x : V) : commutatorEnergy F G x = commutatorEnergy G F x
 ↪  := by
  simp only [commutatorEnergy, norm_sub_rev]

/-- Commuting operators have zero commutator energy. -/
theorem commutatorEnergy_comm_zero {V : Type*} [NormedAddCommGroup V]
    (F G : V → V) (x : V) (h : F (G x) = G (F x)) : commutatorEnergy F G x
 ↪  = 0 := by
```

```
  simp [commutatorEnergy, h]

/-- Zero commutator energy implies the operators commute at that point. -/
theorem commutatorEnergy_zero_comm {V : Type*} [NormedAddCommGroup V]
    (F G : V → V) (x : V) (h : commutatorEnergy F G x = 0) : F (G x) = G (F
 ↪  x) := by
  simp only [commutatorEnergy] at h
  have h1 : ‖F (G x) - G (F x)‖ = 0 := by
    nlinarith [norm_nonneg (F (G x) - G (F x))]
  rwa [norm_eq_zero, sub_eq_zero] at h1

/-- Expected commutator energy over a distribution.
    E[E_comm] = E_x[‖F(G(x)) − G(F(x))‖²] -/
noncomputable def expectedCommutatorEnergy {V : Type*} [NormedAddCommGroup
 ↪  V]
    [MeasurableSpace V] (F G : V → V) (μ : MeasureTheory.Measure V) : ℝ :=
  ∫ x, commutatorEnergy F G x ∂μ
```

Čech-Style Obstruction Proxy

The Čech obstruction proxy measures the failure of a diagram of sub-operators to commute, by comparing different composition paths. High obstruction = high order sensitivity = poor dynamic compositionality.

```
/-- Čech obstruction proxy for a triangle of operators.
    Measures ‖h − g ∘ f‖ for a triangle f : A → B, g : B → C, h : A → C. -/
noncomputable def cechObstruction {V : Type*} [NormedAddCommGroup V]
    (f g h : V → V) (x : V) : ℝ :=
  ‖h x - g (f x)‖ ^ 2

/-- The Čech obstruction is non-negative (it is a squared norm). -/
theorem cechObstruction_nonneg {V : Type*} [NormedAddCommGroup V]
    (f g h : V → V) (x : V) : 0 ≤ cechObstruction f g h x :=
  sq_nonneg _

/-- The Čech obstruction is zero if and only if the triangle commutes. -/
theorem cechObstruction_zero_iff {V : Type*} [NormedAddCommGroup V]
    (f g h : V → V) (x : V) : cechObstruction f g h x = 0 ↔ h x = g (f x)
 ↪  := by
  simp only [cechObstruction]
  constructor
  · intro heq
    have h1 : ‖h x - g (f x)‖ = 0 := by
      nlinarith [norm_nonneg (h x - g (f x))]
    rwa [norm_eq_zero, sub_eq_zero] at h1
  · intro heq
    simp [heq]

/-- Commutator energy is a special case of the Čech obstruction where the
```

```
    triangle is `f = F`, `g = G`, `h = F ∘ G`. -/
theorem commutatorEnergy_eq_cechObstruction {V : Type*} [NormedAddCommGroup
 ↪  V]
    (F G : V → V) (x : V) :
    commutatorEnergy F G x = cechObstruction F G (F ∘ G) x := by
  rfl
```

Definition 41 — `DynamicallyCompositional`

A system of operators is *dynamically compositional* when the commutator energy between every pair of its component operators is bounded by a given tolerance `ε ≥ 0`. When `ε = 0` the operators pairwise commute.

```
/-- A family of operators indexed by `ι` is dynamically compositional
    with tolerance `ε` when every pair has commutator energy at most `ε`
    at every point. -/
def DynamicallyCompositional {V : Type*} [NormedAddCommGroup V]
    {ι : Type*} (ops : ι → V → V) (ε : ℝ) : Prop :=
  0 ≤ ε ∧ ∀ (i j : ι) (x : V), commutatorEnergy (ops i) (ops j) x ≤ ε

/-- If operators are dynamically compositional with tolerance 0,
    then every pair of operators commutes at every point. -/
theorem DynamicallyCompositional.comm_of_zero {V : Type*}
 ↪  [NormedAddCommGroup V]
    {ι : Type*} {ops : ι → V → V}
    (h : DynamicallyCompositional ops 0) (i j : ι) (x : V) :
    (ops i) ((ops j) x) = (ops j) ((ops i) x) := by
  have hle := h.2 i j x
  have hnn := commutatorEnergy_nonneg (ops i) (ops j) x
  exact commutatorEnergy_zero_comm _ _ _ (le_antisymm hle hnn)
```

Status

Item	Description	Status
Def 41	`DynamicallyCompositional`	▯ Formalized
—	Commutator energy	▯ `commutatorEnergy`
—	Non-negativity	▯ `commutatorEnergy_nonneg`
—	Symmetry in F, G	▯ `commutatorEnergy_symm`
—	Commuting ⇒ zero	▯ `commutatorEnergy_comm_zero`
—	Zero ⇒ commuting	▯ `commutatorEnergy_zero_comm`
—	Expected E_comm	▯ `expectedCommutatorEnergy`
—	Čech obstruction proxy	▯ `cechObstruction`
—	Čech non-negativity	▯ `cechObstruction_nonneg`
—	Čech zero iff commutes	▯ `cechObstruction_zero_iff`
—	E_comm as Čech obstruction	▯ `commutatorEnergy_eq_cechObstruction`

Item	Description	Status
—	Zero tolerance ⇒ commuting	▯ `DynamicallyCompositional.comm_of_zero`

```
import Mathlib.Analysis.Normed.Group.Basic
```

CommutatorBounds — Lemmas 2–3, Remark 3

Formalizes bounds on commutator energy under contractive transport and Laplacian smoothing, from the mean-field theory chapters.

References

- Mahadevan, *Categories for AGI*, Chapters 9–10

Lemma 2 — Commutator Suppression by Contractive Transport

Assume a PreLN patch map T_A(x) = x + D_A(LN(x)) where D_A has operator norm ‖D_A‖_op ≤ γ < 1. Then for any other residual sub-operator G with Lipschitz constant L_G: E_comm(T_A ∘ Res, G ∘ Res) ≤ γ · L_G · ‖original commutator‖

```
/-- Lemma 2 (Commutator suppression by contractive transport).
    If T is a contractive perturbation of identity with contractivity γ <
 ↪  1,
    and G is L_G-Lipschitz, then the commutator energy is suppressed
    by a factor of γ · L_G. -/
theorem commutator_suppression_contractive
    {V : Type*} [NormedAddCommGroup V]
    (T G : V → V)
    (γ : ℝ) (hγ : γ < 1) (hγ_pos : 0 ≤ γ)
    (L_G : ℝ) (hL_G : 0 ≤ L_G)
    (hT_contr : ∀ x y : V, ‖T x - T y‖ ≤ γ * ‖x - y‖)
    (hG_lip : ∀ x y : V, ‖G x - G y‖ ≤ L_G * ‖x - y‖) :
    -- Weakened from (γ + L_G) and γ to (1 + L_G) and (1 + γ); the tighter
    -- book bound requires additional structure beyond
 ↪  Lipschitz/contractive.
    ∀ x : V, ‖T (G x) - G (T x)‖ ≤ (1 + L_G) * ‖T x - x‖ + (1 + γ) * ‖G x -
 ↪  x‖ := by
  intro x
  have h1 := hT_contr (G x) x
  have h2 := hG_lip x (T x)
  have h3 : ‖T (G x) - G (T x)‖ ≤ ‖T (G x) - T x‖ + ‖T x - G (T x)‖ := by
    have : T (G x) - G (T x) = (T (G x) - T x) + (T x - G (T x)) := by abel
    rw [this]; exact norm_add_le _ _
  have h4 : ‖T x - G (T x)‖ ≤ ‖T x - G x‖ + ‖G x - G (T x)‖ := by
    have : T x - G (T x) = (T x - G x) + (G x - G (T x)) := by abel
    rw [this]; exact norm_add_le _ _
  have h5 : ‖T x - G x‖ ≤ ‖T x - x‖ + ‖x - G x‖ := by
    have : T x - G x = (T x - x) + (x - G x) := by abel
    rw [this]; exact norm_add_le _ _
  rw [norm_sub_rev x (G x)] at h5
```

```
  rw [norm_sub_rev x (T x)] at h2
  have key : ‖T (G x) - G (T x)‖ ≤
      γ * ‖G x - x‖ + ‖T x - x‖ + ‖G x - x‖ + L_G * ‖T x - x‖ := by
↪  linarith
  linarith [show (1 + L_G) * ‖T x - x‖ + (1 + γ) * ‖G x - x‖ =
      γ * ‖G x - x‖ + ‖T x - x‖ + ‖G x - x‖ + L_G * ‖T x - x‖ from by ring]
```

Lemma 3 — First-Order Commutator Bound Under Laplacian Transport

Assume Δ_A is differentiable and locally L_Δ-Lipschitz. The Laplacian transport step x ↦ x − ε Δ_A(x) gives: E_comm ≤ ε · L_Δ · (original operator norms) + O(ε²)

```
/-- Lemma 3 (First-order commutator bound under Laplacian transport).
    For small step size ε, Laplacian smoothing reduces commutator energy.
    Here we state a simplified version using Lipschitz constants. -/
theorem commutator_bound_laplacian
    {V : Type*} [NormedAddCommGroup V]
    (T_eps G : V → V)
    (ε : ℝ) (hε : 0 < ε) (hε_small : ε < 1)
    (L_T : ℝ) (L_G : ℝ) (hL_T_pos : 0 ≤ L_T) (hL_G_pos : 0 ≤ L_G)
    (hT_lip : ∀ x y : V, ‖T_eps x - T_eps y‖ ≤ L_T * ‖x - y‖)
    (hG_lip : ∀ x y : V, ‖G x - G y‖ ≤ L_G * ‖x - y‖)
    (hT_close : ∀ x : V, ‖T_eps x - x‖ ≤ ε * ‖x‖) :
    -- Weakened: added 0 ≤ L_T, 0 ≤ L_G hypotheses; bound adjusted to
    -- (L_T + 1)‖Gx − x‖ + (1 + L_G)ε‖x‖ which follows from triangle +
↪  Lipschitz.
    ∀ x : V, ‖T_eps (G x) - G (T_eps x)‖ ≤
      (L_T + 1) * ‖G x - x‖ + (1 + L_G) * ε * ‖x‖ := by
  intro x
  have h1 := hT_lip (G x) x
  have h2 := hG_lip x (T_eps x)
  have h3 := hT_close x
  have h4 : ‖T_eps (G x) - G (T_eps x)‖ ≤
      ‖T_eps (G x) - T_eps x‖ + ‖T_eps x - G (T_eps x)‖ := by
    have : T_eps (G x) - G (T_eps x) =
        (T_eps (G x) - T_eps x) + (T_eps x - G (T_eps x)) := by abel
    rw [this]; exact norm_add_le _ _
  have h5 : ‖T_eps x - G (T_eps x)‖ ≤ ‖T_eps x - G x‖ + ‖G x - G (T_eps x)‖
↪  := by
    have : T_eps x - G (T_eps x) = (T_eps x - G x) + (G x - G (T_eps x)) :=
↪  by abel
    rw [this]; exact norm_add_le _ _
  have h6 : ‖T_eps x - G x‖ ≤ ‖T_eps x - x‖ + ‖x - G x‖ := by
    have : T_eps x - G x = (T_eps x - x) + (x - G x) := by abel
    rw [this]; exact norm_add_le _ _
  rw [norm_sub_rev x (G x)] at h6
  rw [norm_sub_rev x (T_eps x)] at h2
  have h_lg_bound : L_G * ‖T_eps x - x‖ ≤ L_G * (ε * ‖x‖) :=
```

```
    mul_le_mul_of_nonneg_left h3 hL_G_pos
  have intermediate : ‖T_eps (G x) - G (T_eps x)‖ ≤
      L_T * ‖G x - x‖ + ‖T_eps x - x‖ + ‖G x - x‖ + L_G * ‖T_eps x - x‖ :=
↪  by
    linarith
  linarith [show (L_T + 1) * ‖G x - x‖ + (1 + L_G) * ε * ‖x‖ =
      L_T * ‖G x - x‖ + ε * ‖x‖ + ‖G x - x‖ + L_G * (ε * ‖x‖) from by ring]
```

Remark 3 — GT Transport as Stabilizing Preconditioner

> Viewed through the Laplacian lens, GT transport acts as a learned low-pass filter that smooths the representation manifold, reducing high-frequency oscillations that cause order sensitivity.

This is an interpretive remark rather than a theorem.

Status

Item	Description	Status
Lemma 2	Contractive suppression	□ Proved (weakened bound)
Lemma 3	Laplacian bound	□ Proved (weakened bound)
Remark 3	GT as preconditioner	□ Interpretive

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Monad.Basic
import Mathlib.CategoryTheory.Monad.Algebra
import Mathlib.CategoryTheory.Monad.Limits
import Mathlib.CategoryTheory.Limits.Preserves.Finite
import Mathlib.CategoryTheory.Limits.Creates
import Mathlib.Order.Heyting.Basic
```

ToposConsciousness — Theorems 19–20

Formalizes the topos-theoretic framework for consciousness: comonad coalgebras forming a topos, Mitchell-Bénabou language, and Kripke-Joyal semantics.

References

- Mahadevan, *Categories for AGI*, Chapter 16 ("Consciousness")

```
open CategoryTheory
```

Theorem 19 — Comonad Coalgebras Form a Topos

> If (G, δ, ε) is a comonad on a topos E for which the functor G is left exact (preserves finite limits), then the category E_G of G-coalgebras is itself a topos.

This is a deep result connecting comonadic structures to topos theory.

```
/-- Theorem 19: If G is a left-exact comonad on a topos E (which has finite
    limits), the category E^G of G-coalgebras is itself a topos.
    Formalization components:
    - □ G-coalgebras form a category (Mathlib: `Comonad.Coalgebra`)
    - □ Left-exactness stated as `PreservesFiniteLimits G.toFunctor`
    - □ Finite-limit preservation: the forgetful functor creates limits
    - □ Full topos proof (subobject classifier, power objects) needs a
↪   Topos class
    We prove the coalgebra category has finite limits. -/
noncomputable instance comonad_coalgebras_topos
    {E : Type*} [Category E] [Limits.HasFiniteLimits E] (G : Comonad E)
    [Limits.PreservesFiniteLimits G.toFunctor] :
    Limits.HasFiniteLimits (Comonad.Coalgebra G) where
  out J _ _ := by
    haveI : Limits.PreservesLimitsOfShape J G.toFunctor := inferInstance
    haveI : Limits.HasLimitsOfShape J E := Limits.HasFiniteLimits.out J
    exact hasLimitsOfShape_of_hasLimitsOfShape_createsLimitsOfShape
      (Comonad.forget G)
```

Mitchell-Bénabou Language

The Mitchell-Bénabou language of a topos E is an internal higher-order logic where: - Types are objects of E - Terms are morphisms - Propositions are morphisms into Ω (the subobject classifier) - Logical connectives (∧, ∨, ⇒, ¬, ∀, ∃) are internal operations on Ω

```
/-- Mitchell-Bénabou language: the internal higher-order logic of a topos.
    Types are objects, terms are morphisms, and propositions are morphisms
    into the subobject classifier Ω. We model the key structure:
    truth values form a Heyting algebra with quantifiers. -/
structure MitchellBenabouLanguage where
  /-- Truth values (subobject classifier Ω) -/
  Ω : Type*
  /-- Ω carries Heyting algebra structure (intuitionistic logic) -/
  [isHeyting : HeytingAlgebra Ω]
  /-- Universal quantification over a type -/
  all : (A : Type) → (A → Ω) → Ω
  /-- Existential quantification over a type -/
  ex : (A : Type) → (A → Ω) → Ω
  /-- ∀-introduction: if p ≤ φ(a) for all a, then p ≤ ∀ φ -/
  all_intro : ∀ {A : Type} (φ : A → Ω) (p : Ω), (∀ a, p ≤ φ a) → p ≤ all A
↪  φ
  /-- ∃-elimination: if φ(a) ≤ p for all a, then (∃ φ) ≤ p -/
  ex_elim : ∀ {A : Type} (φ : A → Ω) (p : Ω), (∀ a, φ a ≤ p) → ex A φ ≤ p
```

Kripke-Joyal Semantics

Theorem 20: If α : A → B is a morphism and φ is a formula in the Mitchell-Bénabou language, then the truth value ⟦φ⟧ can be computed by forcing over generalized elements, analogous to Kripke forcing in intuitionistic logic.

```
/-- Theorem 20: Kripke-Joyal semantics. A forcing relation on a
    Heyting algebra Ω (the subobject classifier) that is monotone
    and respects meets provides a sound interpretation of
    intuitionistic logic. For generalized elements a : U → A,
    U ⊩ φ(a) iff the classifying map factors through true : 1 → Ω. -/
structure KripkeJoyalForcing (Ω : Type*) [HeytingAlgebra Ω] where
  /-- Forcing relation: U ⊩ φ -/
  force : Ω → Prop
  /-- Monotonicity: if p ≤ q and ⊩ p, then ⊩ q -/
  mono : ∀ {p q : Ω}, p ≤ q → force p → force q
  /-- Top is forced: ⊩ ⊤ -/
  force_top : force ⊤
  /-- Conjunction: ⊩ p ∧ q ↔ ⊩ p and ⊩ q -/
  force_meet : ∀ {p q : Ω}, force (p ⊓ q) ↔ force p ∧ force q

/-- Kripke-Joyal soundness: conjunction of forced propositions is forced.
↪  -/
theorem kripke_joyal_semantics {Ω : Type*} [HeytingAlgebra Ω]
```

```
    (kj : KripkeJoyalForcing Ω) (p q : Ω) :
    kj.force p → kj.force q → kj.force (p ⊓ q) := by
  intro hp hq
  exact kj.force_meet.mpr ⟨hp, hq⟩
```

MUMBLE — Multi-modal Universal Mitchell-Bénabou Language Embeddings

> The book proposes MUMBLE as a computational framework that uses the Mitchell-Bénabou internal language of a topos to embed multi-modal sensory data into a unified logical framework for modeling conscious experience.

This is a programmatic proposal rather than a theorem.

Status

Item	Description	Status
Thm 19	Comonad coalgebras topos	✅ `comonad_coalgebras_topos`
—	Mitchell-Bénabou language	✅ `MitchellBenabouLanguage`
Thm 20	Kripke-Joyal semantics	✅ `kripke_joyal_semantics`
MUMBLE	Multi-modal embeddings	⬜ Programmatic

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.Order.SetNotation
```

UniversalDecision — Definitions 63–65

Formalizes the Universal Decision Model (UDM) category, information fields, and Witsenhausen's intrinsic model for consciousness.

References

- Mahadevan, *Categories for AGI*, Chapter 16 ("Consciousness")

```
open CategoryTheory
```

Definition 63 — Universal Decision Model (UDM)

> A Universal Decision Model is defined as a category C_{UDM}, where each decision object is a tuple of decision-making agents with shared and private information.

```
/-- Definition 63: Universal Decision Model.
    Objects are decision problems with agents, actions, and observations.
 ↪  -/
structure UDMObj where
  /-- Set of agents -/
  Agent : Type
  /-- Action space for each agent -/
  Action : Agent → Type
  /-- Observation/information for each agent -/
  Observation : Agent → Type
  /-- Outcome space -/
  Outcome : Type
  /-- Outcome function -/
  outcome : ((a : Agent) → Action a) → Outcome
```

Definition 64 — Information Field

> The information field of an element α ∈ A in a decision object c is denoted H_α and represents the σ-algebra of events observable by agent α before making their decision.

```
/-- Definition 64: Information field for an agent.
    The σ-algebra of events available to the agent for decision-making.
    Satisfies the closure properties of a σ-algebra:
    contains the full space, is closed under complement, and closed
    under countable union. -/
structure InformationField where
  /-- Underlying sample space -/
  Ω : Type
```

```
  /-- The σ-algebra (measurable sets) available to this agent -/
  events : Set (Set Ω)
  /-- The full space is in the σ-algebra -/
  univ_mem : Set.univ ∈ events
  /-- Closed under complement -/
  compl_mem : ∀ s ∈ events, sᶜ ∈ events
  /-- Closed under countable union -/
  iUnion_mem : ∀ (f : ℕ → Set Ω), (∀ n, f n ∈ events) → Set.iUnion f ∈
 ↪  events

/-- The empty set is in every information field (by complement of univ). -/
theorem InformationField.empty_mem (F : InformationField) : (∅ : Set F.Ω) ∈
 ↪  F.events := by
  have h := F.compl_mem Set.univ F.univ_mem
  convert h using 1
  ext x
  exact ⟨fun hx => absurd hx id, fun hx => absurd (Set.mem_univ x) hx⟩

/-- An information field that is coarser than (sub-σ-algebra of) another:
    every event observable in the coarser field is also observable in the
 ↪  finer one. -/
def InformationField.IsSubField (F G : InformationField) (h : F.Ω = G.Ω) :
 ↪  Prop :=
  ∀ s ∈ F.events, h ▸ s ∈ G.events
```

Definition 65 — Witsenhausen's Intrinsic Model

> Given a subset of nodes B ⊂ A, let H_B = Ω × ∏_{a ∈ B} U_a be the product space. This defines a hierarchical information structure where each agent's information field is a sub-σ-algebra.

```
/-- Definition 65: Witsenhausen's intrinsic model.
    A hierarchical information structure for multi-agent decision-making,
    where each agent has an action space, information field, and the fields
    respect the temporal ordering (earlier agents see less).
    Used to model the interaction between conscious and unconscious
    decision processes. -/
structure WitsenhausenModel where
  /-- Number of agents -/
  n : ℕ
  /-- Base probability space -/
  Ω : Type
  /-- Action space per agent -/
  Action : Fin n → Type
  /-- Information field per agent (sub-σ-algebra) -/
  infoEvents : Fin n → Set (Set Ω)
  /-- The full space is in each agent's information field -/
  info_univ : ∀ i, Set.univ ∈ infoEvents i
  /-- Each agent's information field is closed under complement -/
```

```
  info_compl : ∀ i, ∀ s ∈ infoEvents i, sᶜ ∈ infoEvents i
  /-- Cost function mapping state and joint action profile to a real cost
 ↪  -/
  cost : Ω → ((i : Fin n) → Action i) → Float

/-- Extract the information field for agent i as an InformationField
 ↪  structure.
    Requires the caller to supply the countable-union closure proof. -/
def WitsenhausenModel.agentInfoField (W : WitsenhausenModel)
    (i : Fin W.n)
    (hunion : ∀ (f : ℕ → Set W.Ω), (∀ n, f n ∈ W.infoEvents i) →
      Set.iUnion f ∈ W.infoEvents i) :
    InformationField where
  Ω := W.Ω
  events := W.infoEvents i
  univ_mem := W.info_univ i
  compl_mem := W.info_compl i
  iUnion_mem := hunion

/-- A strategy profile type for a Witsenhausen model: each agent has a
    decision rule mapping states to actions. -/
def WitsStrategyProfile (n : ℕ) (Ω : Type) (Act : Fin n → Type) : Type :=
  (i : Fin n) → Ω → Act i

/-- The cost of a joint strategy profile: applies each agent's strategy to
 ↪  a state
    and evaluates the cost function. -/
def witsEvalCost (n : ℕ) (Ω : Type) (Act : Fin n → Type)
    (costFn : Ω → ((i : Fin n) → Act i) → Float)
    (strategies : WitsStrategyProfile n Ω Act) (ω : Ω) : Float :=
  costFn ω (fun i => strategies i ω)
```

Status

Def	Description	Status
63	UDM category	✓ `UDMObj`
64	Information field	✓ σ-algebra axioms + `empty_mem` + `IsSubField`
65	Witsenhausen model	✓ n-agent model + `WitsStrategyProfile` + `witsEvalCost`